

Smart Car System

CASE STUDY: STM32F401VE

A five-node distributed embedded system combining Proteus simulation, Keil development, STM32CubeMX configuration, and UART networking, featuring an STM32Cube.AI-powered emergency braking module with real-time, on-screen obstacle alerts.

Smart Car System Design • **Team Project — 5 Members** • Proteus + Keil + CubeMX + STM32Cube.AI

| Presentation Agenda

INTRODUCTION DECK — ROADMAP

- 1 **Project Overview** — goal, scope, deliverables
- 2 **Toolchain** — Proteus Simulation, Keil μ Vision, STM32CubeMX + STM32Cube.AI
- 3 **Hardware Platform** — Why STM32F401VE?
- 4 **System Architecture** — 5 nodes, Master–Slave
- 5 **Communication Layer** — UART + COMPIM
- 6 **Web Dashboard** — HTML + CSS + Java Script + Node.js + WebSocket
- 7 **Git Organization** — repos & submodules

| Project Overview

DISTRIBUTED EMBEDDED SYSTEM — SMART CAR

- **Goal:** Build a modular smart car simulation where five microcontrollers cooperate to monitor safety, environment, speed, battery and AI-based obstacle detection.
- **Approach:** Each team member owns one node. Nodes exchange data over UART inside Proteus
- **Master node** aggregates all telemetry and drives a graphical LCD plus a real-time web dashboard.
- **AI node** runs a TinyML decision layer to detect obstacles and trigger both on-screen warnings and autonomous emergency braking.

| Development Toolchain

PROTEUS + KEIL MVISION + STM32CUBEMX



Proteus

Full circuit simulation — MCUs, sensors, virtual COM (COMPIM), LCD, motors and UART wiring between all five nodes.



Keil μVision

C firmware development, ARM compilation, debugging and generating the .hex images loaded into each MCU.



STM32CubeMX & STM32Cube.AI

Pin multiplexing, peripheral init (USART/ADC/TIM) — auto-generates HAL code and integrates optimized TinyML obstacle detection models

WORKFLOW

CubeMX configures the peripherals → **Keil** compiles the firmware → **Proteus** simulates the hardware and wires the 5 nodes together over UART.

| Hardware Platform — STM32F401VE

THE MCU BEHIND EVERY NODE

Core	ARM Cortex-M4F @ 84 MHz
Flash	512 KB
SRAM	96 KB
FPU / DSP	Single-precision FPU + DSP
ADC	1 × 12-bit, up to 16 channels
Timers	7 × 16-bit + 2 × 32-bit + PWM
USART / UART	3 × USART
SPI / I ² C	4 × SPI · 3 × I ² C

WHY F401VE FOR THIS PROJECT?

TinyML & Hardware FPU: Cortex-M4F single-precision FPU + DSP enables efficient, local edge-AI inference on the node.

Proteus USART Limit & MUX: F401VE has only 3 USARTs. We designed a dynamic MUX topology to network all 4 nodes without SPI overhead.

Rich Timer Set: 9 hardware timers easily handle PWM lighting, speed simulation, and interrupts concurrently.

Stable Emulation & HAL: One of the few powerful STM32s with a robust Proteus VSM simulation model and native CubeMX support.

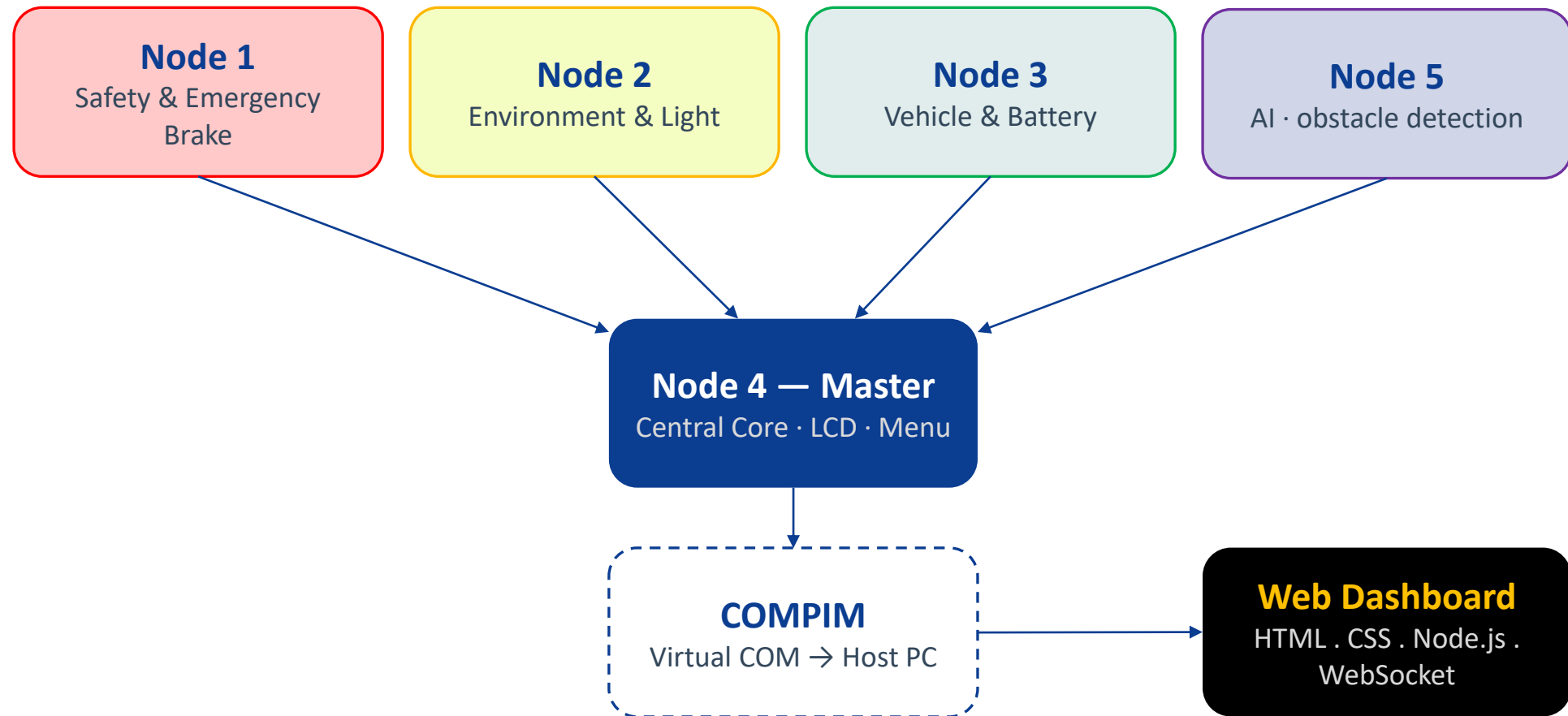
System Architecture

PART II

Five specialized MCUs operating in a Master–Slave network, bridged to a real-time web dashboard — featuring an integrated Edge-AI engine for autonomous braking and obstacle detection.

| Overall Architecture

MASTER-SLAVE · UART NETWORK · COMPIM BRIDGE



| The Five Nodes at a Glance

EACH NODE = ONE TEAM MEMBER = ONE STM32F401VE

NODE 1

Safety System

- TPMS (Tire Pressure)
- Collision Sensor
- Emergency Brake Control
- Safety Data Transmission (UART/SPI)
- Speed Limitation State Machine

NODE 2

Environment & Light

- LDR Ambient Sensor
- PWM Headlight Control
- LM35 temperature
- Auto light mode
- Automatic Windshield Wiper Control

NODE 3

Vehicle & Battery Monitoring

- Battery voltage (ADC)
- Speed via TIM/IRQ
- Odometer count
- Start / Park Management
- Telemetry packets

NODE 4

Master Core & LCD

- Central Data Aggregator
- Character/Graphical LCD Interface
- User menu / GUI
- UART hub for 1-2-3-5
- System Monitoring

NODE 5

AI & Braking

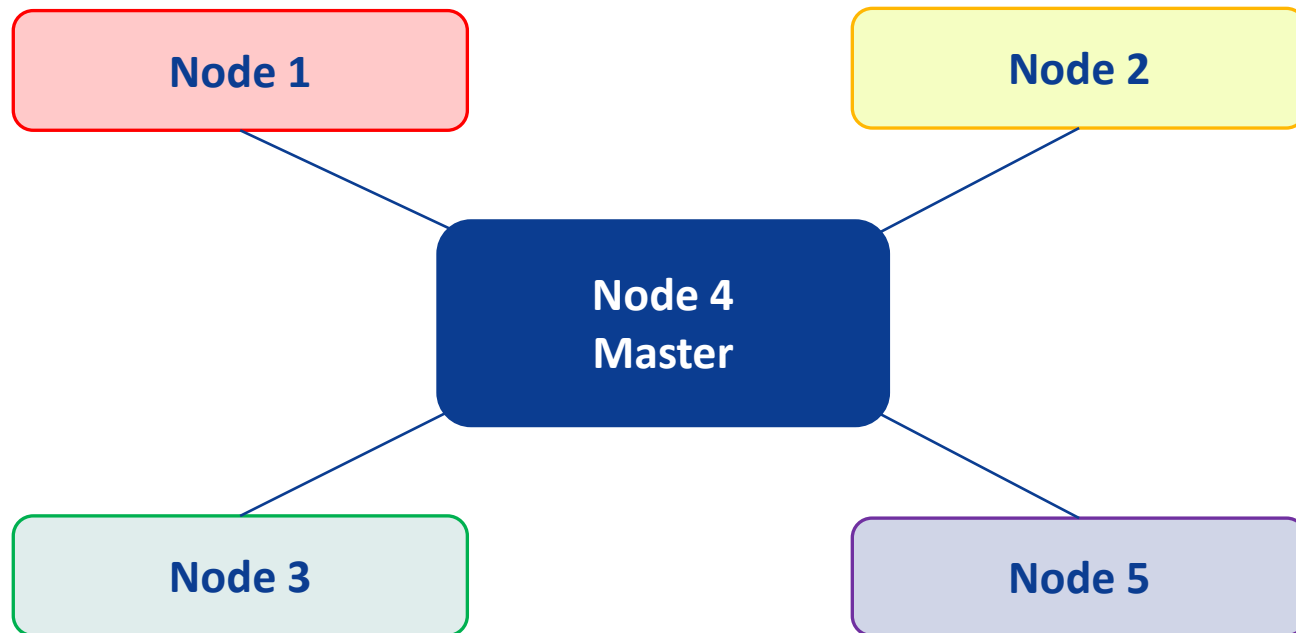
- TinyML Inference Engine
- Obstacle detection
- Decision FSM
- Emergency Brake Trigger
- Decision Logic & Speed Control Commands

OWNERSHIP

Every member is responsible for their node's **Keil project**, **CubeMX config**, **Proteus schematic** and a **dedicated GitHub repository** — later all merged as submodules into the central Proteus project.

| Inter-MCU Communication

UART WIRING BETWEEN ALL FIVE NODES IN PROTEUS



Network Topology & Physical Layer

- The Proteus USART Constraint: Although STM32F401VE is powerful, Proteus limits its simulation model to only 3 hardware USARTs.
- The MUX Solution: To connect all 5 nodes, we implemented a Multiplexer (MUX) Hardware Topology in Proteus to dynamically route the TX/RX lines of the Slaves (Nodes 1, 2, 3, 5) to the Master (Node 4).
- Serial Config: All nodes are synchronized on USART @ 9600 bps, 8N1 (8 data bits, no parity, 1 stop bit).

Protocol & Framing Layer

- Framing Protocol: Asynchronous, human-readable ASCII-framed messages terminated by `\r\n` (CRLF) to ensure robust packet boundary delineation.
- Telemetry Format (Slave → Master): Self-describing tokenized patterns with explicit labels (e.g., "Speed=%d km/h Battery=%d %% " or "Temp:%dC | Lights PWM:%d").
- Parsing: `strstr()` for alerts, `sscanf()` for data extraction.
- Memory Mapping: Master dynamically parses incoming lines via UART interrupts and maps them directly into node-specific global variables (`car_speed`, `tire_pressure`, etc.).

| Real-Time Web Dashboard

STM32 → COMPIM → NODE.JS → WEBSOCKET → BROWSER



- Packet: The master serializes all node states as one JSON string per cycle.
- COMPIM: Proteus component that pipes MCU UART to the host's virtual COM port.
- Node.js: Reads the COM port with serialport and broadcasts to browsers with ws.

DASHBOARD CARDS

- Safety: Tires · Brake · Alerts
- Environment: Temp · Lights · Status
- Status: Speed (km/h) · Battery (%)
- AI: Obstacle Distance · Path Clear

| Team Workflow & Git Organization

MULTI-REPO ARCHITECTURE & INTEGRATION STRATEGY

[HTTPS://GITHUB.COM/SMART-CAR-DEV](https://github.com/smart-car-dev)

Repository Structure (7 Repositories in Total)

MCU/Node Repositories: Each team member owns and develops a dedicated node containing Keil source code, CubeMX .ioc configuration, and localized testbenches:

- node-1-safety
- node-2-env
- node-3-status
- node-4-master
- node-5-ai
- Dashboard Repository: smart-car-dashboard
- Central Simulation & Integration Repository: SmartCar_Simulation

The "Umbrella" repository that integrates the 5 node repos as Git Submodules to keep all codebase in one synchronized place. Houses the final, unified Proteus schematic (.pdsprj) connecting all 5 virtual STM32 microcontrollers, virtual terminals, and the COMPIM module.

| Project Roadmap & Current Status

WHERE WE ARE TODAY / FUTURE WORKS

Completed Core

DONE

- Distributed Network: Active 5-node UART communication in Proteus using a dynamic MUX hardware topology.
- Basic Telemetry: Real-time speed simulation, battery ADC reading, LM35 temperature monitoring, and LDR auto-headlights (PWM).
- AI Core: TinyML obstacle detection inference runs successfully on Node 5.
- Web UI: Live web dashboard streaming real-time telemetry via COMPIM + Node.js middleware.

Future Work & Advanced Integration

NEXT

- AI Braking Loop: Automatic braking via TinyML obstacle detection.
- Automated Logic: TPMS-based speed limiting, rain-controlled wipers, and ignition safety checks.
- Graphical GUI: Upgrade Node 4 to a graphical LCD interface.
- Bidirectional Control: Enable Master-to-Node command frames and manual overrides.
- Remote Web Control: Control lights, wipers, and emergency functions from the web dashboard.
- Cloud IoT: Migrate from local Node.js to AWS IoT / Adafruit IO for remote monitoring and telemetry.

Individual Node Walkthrough

PART III · NODE DEEP-DIVE

Now, each team member will present a deep dive into their specialized microcontroller node. We will walk you through the firmware logic, hardware peripherals, and specific challenges of all five sub-systems, demonstrating how they cooperate seamlessly to form one intelligent vehicle network.

Node 1: Safety & Emergency Brake Node

PRESENTER: HOSSEIN RANJBARI

The safety node continuously monitors tire pressure and reacts to a collision within milliseconds, then reports its state to the master over UART — all in a fully non-blocking architecture.

| Node 1 — Mission & Responsibilities

WHAT THIS NODE MUST DO INSIDE THE SMART CAR

Collision Detection & Emergency Response

- Hardware interrupt-based collision detection.
- Immediate emergency brake activation and status LED updates.

Tire Pressure Monitoring

- TPMS monitoring every 500 ms using ADC.
- Real-time low-pressure indication via dedicated TPMS LED.

Telemetry & Communication

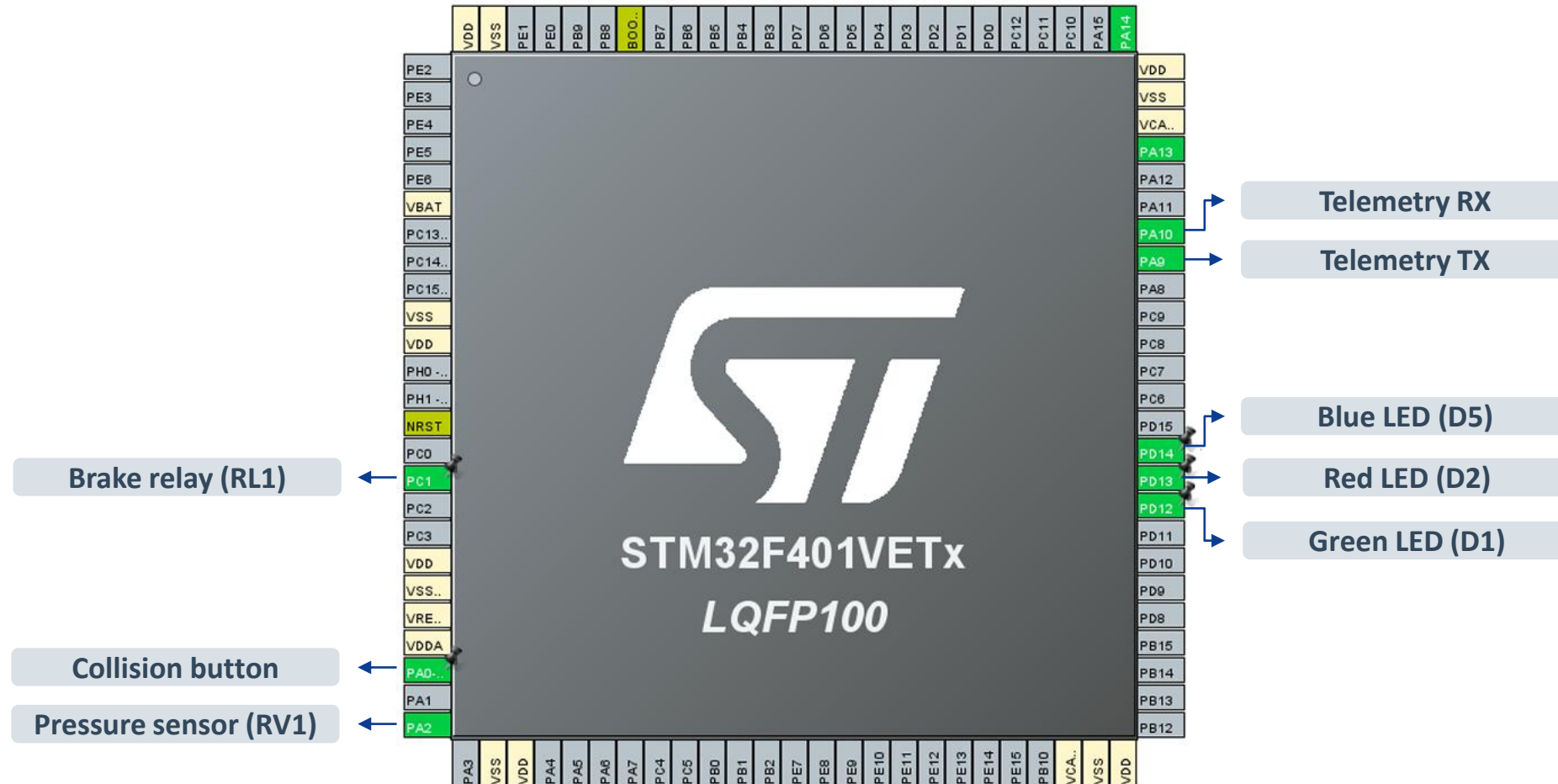
- Telemetry transmission to the Master Node every 500 ms.
- Emergency-tagged messages during critical events.

Real-Time System Architecture

- Emergency events prioritized over normal operations.
- HAL_GetTick() scheduling with interrupt-driven UART communication.

| STM32CubeMX Configuration

PERIPHERAL SETUP, PIN MAPPING, AND SYSTEM INITIALIZATION



| Three Development Phases

EACH PHASE ADDED ONE LAYER TO THE SAME FIRMWARE

Phase1

Emergency Brake

Hardware interrupt on EXTI0 (PA0) triggers the emergency response sequence and immediately activates the brake relay and status indicators.

Falling-edge detection eliminates the need for polling, ensuring that the MCU prioritizes safety events at all times.

Emergency braking is performed through GPIO-controlled relay actuation, providing a hardware-level safety mechanism for the Smart Car system

phase2

TPMS Monitoring

Tire pressure is simulated using a potentiometer connected to ADC1_IN2 (PA2), providing an adjustable analog input for testing.

The ADC continuously samples the input value and compares it against a predefined safety threshold (2000 counts).

When the pressure drops below the threshold, the TPMS LED (PD14) is activated

phase3

UART Telemetry

USART1 @ 9600 8N1 on PA9 transmits framed telemetry messages to the Master Node using HAL_UART_Transmit_IT(). Data packets contain TPMS readings and emergency status information.

Node 1

Safety & Emergency Brake

Emergency Brake — immediate hardware interrupt response.

PHASE 1

EXTIO on PA0 is configured for falling-edge detection with NVIC priority 0. No polling loops or blocking delays are involved. The instant a collision signal pulls PA0 low, the interrupt service routine takes control, engages the brake relay, and updates the system status indicators.

| EXTI Interrupt Logic

GPIO_EXTI0 · FALLING EDGE · NVIC PRIORITY 0

```
void
HAL_GPIO_EXTI_Callback(uint16_t
GPIO_Pin){
    if(GPIO_Pin == GPIO_PIN_0){
        // Green OFF, Red ON, Relay
ON
        ...
        brake_emergency_flag    = 1;
        brake_emergency_active = 1;
    }
}
```

WHY HARDWARE INTERRUPT?

A polling loop could delay the emergency response. The MCU immediately executes the ISR when PA0 transitions low.

TWO FLAGS, TWO JOBS

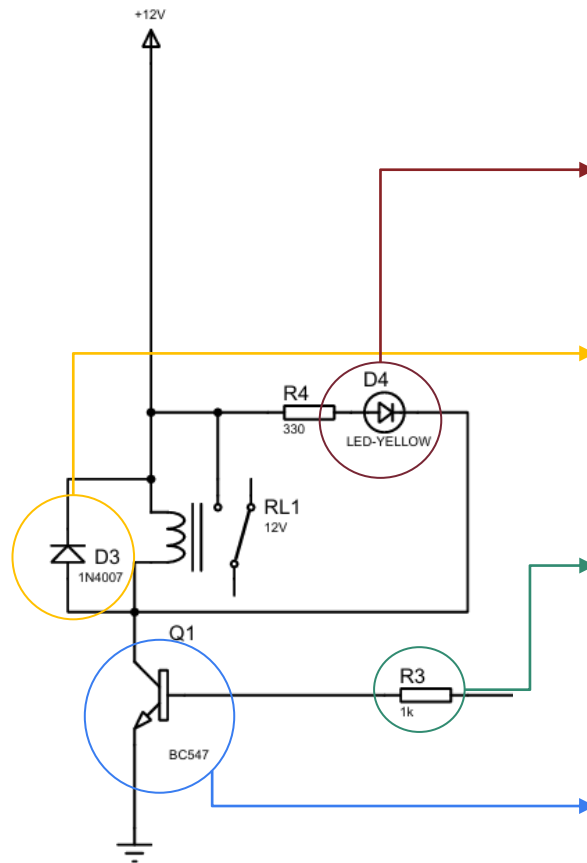
brake_emergency_active keeps telemetry on the emergency path, while brake_emergency_flag is reserved for event handling or future one-shot logic.

LATCHED BRAKE

Once activated, the brake relay remains engaged for the remainder of execution unless the system is reset.

| Relay Driver Circuit

MCU → BC547 → RL1 (12 V) WITH 1N4007 FLYBACK DIODE



THE BRAKE INDICATOR (D4 – STATUS LED)

Provides a visual indication that the emergency braking circuit has been activated during simulation.

THE PROTECTION STAGE (D3 – 1N4007)

A flyback diode connected across the relay coil suppresses back-EMF and protects the transistor from inductive voltage spikes.

THE CURRENT LIMITER (R3 – 1KΩ)

The base resistor limits transistor input current and protects the STM32 GPIO pin during switching operations.

THE SWITCH (BC547 NPN TRANSISTOR)

The STM32 drives the brake relay using a 3.3 V GPIO output. Operating as a low-side switch, the BC547 safely controls the 12 V relay coil.

TPMS — reading tire pressure through the ADC.

PHASE 2

ADC1 · Channel 2 · 12-bit · 480-cycle sampling · 500 ms scheduled sampling.

| ADC1 Configuration

CUBEMX SETTINGS FOR ADC1_IN2 ON PA2

PARAMETER	VALUE
Instance	ADC1
Channel	IN2 · PA2
Resolution	12-bit (0 – 4095)
Alignment	Right
Scan mode	Disabled (single channel)
Continuous	Disabled — software start
Sampling time	480 cycles
Prescaler	PCLK2 / 2

WHY 480 CYCLES?

The maximum sampling time — improves stability of readings under Proteus simulation and reduces noise from the potentiometer source impedance.

THRESHOLD

TPMS_THRESHOLD = 2000. Below → alert LED; above → normal.

POLL CADENCE

TPMS_CHECK_PERIOD_MS = 500. Enough for a mechanical sensor, cheap for the CPU.

| ADC-Based TPMS Monitoring

CHECK_TIRE_PRESSURE() CALLED EVERY 500 MS FROM MAIN

```
void Check_Tire_Pressure(void)
{
    HAL_ADC_Start(&hadc1);

    if (HAL_ADC_PollForConversion(&hadc1, 10) ==
    HAL_OK)
    {
        tire_pressure_adc = HAL_ADC_GetValue(&hadc1);

        if (tire_pressure_adc < TPMS_THRESHOLD)
            HAL_GPIO_WritePin(TPMS_LED_GPIO_Port,
                              TPMS_LED_Pin,
GPIO_PIN_SET);
        else
            HAL_GPIO_WritePin(TPMS_LED_GPIO_Port,
                              TPMS_LED_Pin,
GPIO_PIN_RESET);
    }

    HAL_ADC_Stop(&hadc1);
}
```

Network Topology & Physical Layer

Check_Tire_Pressure() is executed every 500 ms using HAL_GetTick().

ADC TIMEOUT PROTECTION

HAL_ADC_PollForConversion(...,10) limits the ADC wait time to 10 ms.

PROTEUS SIMULATION

RV1 emulates the tire pressure sensor for ADC threshold testing.

Telemetry — UART link to the master node.

PHASE 3

USART1 @ 9600 8N1 · PA9 TX · framed ASCII strings · interrupt-driven transmit.

| ADC-Based TPMS Monitoring

CHECK_TIRE_PRESSURE() CALLED EVERY 500 MS FROM MAIN

PARAMETER	VALUE
Instance	USART1
Mode	Asynchronous · TX/RX
Baud rate	9600 bps
Word length	8 bits
Stop bits	1
Parity	None
Flow control	None
Oversampling	16
TX pin	PA9

Why 9600 Baud?

A common communication standard across all Smart Car nodes. Reliable in Proteus and compatible with the Virtual Terminal.

WHY INTERRUPT-DRIVEN TX?

HAL_UART_Transmit() blocks the CPU, while HAL_UART_Transmit_IT() returns immediately, allowing TPMS monitoring and EXTI interrupts to remain responsive

MASTER INTERFACE

The Master Node receives telemetry from USART1_TX (PA9) using the same baud rate and framing configuration.

| Message Format

TWO FRAME FLAVOURS: NORMAL TELEMETRY VS EMERGENCY

Normal Frame (every 500 ms)

Tire Pressure: n\r\3000

Emergency Frame

Tire Pressure: 1250 EMERGENCY\r\n

Frame Structure

ASCII-based telemetry messages

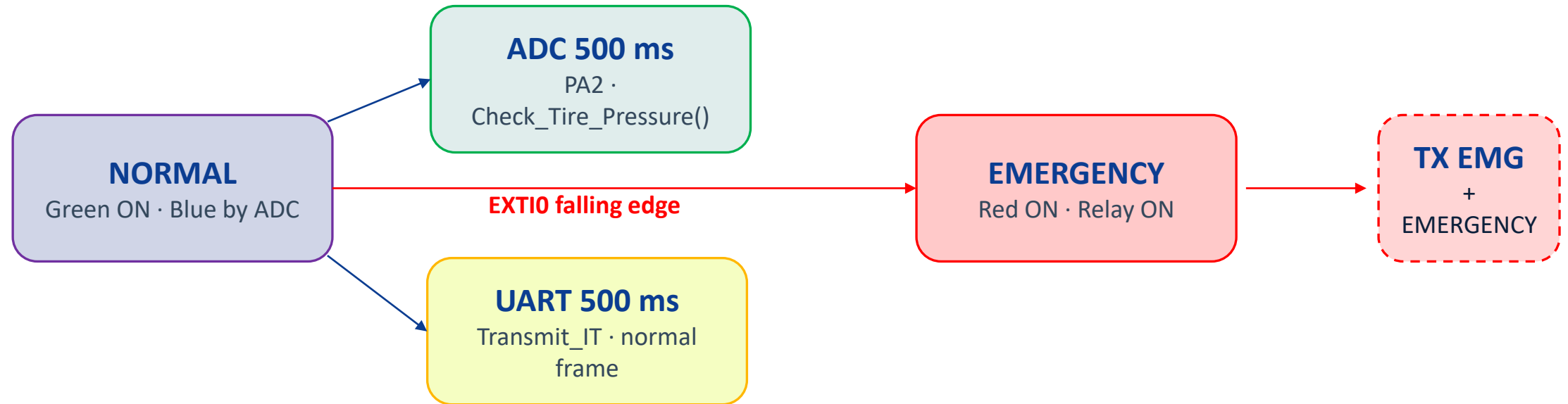
Every packet terminates with \r\n

**The Master Node detects the
EMERGENCY keyword and extracts
the TPMS value.**

```
if (brake_emergency_active)
{
    sprintf(buffer,
        "Tire Pressure: %u EMERGENCY\r\n",
        tire_pressure_adc);
}
else
{
    sprintf(buffer,
        "Tire Pressure: %u\r\n",
        tire_pressure_adc);
}
```

| Runtime Behaviour — State Flow

HOW EXTI, ADC AND UART COOPERATE INSIDE MAIN()



KEY PROPERTY

Once `brake_emergency_active` is asserted, all UART telemetry is permanently switched to the emergency frame until the system is reset.

| Key Takeaways from

WHAT WE WANT THE AUDIENCE TO REMEMBER

- No HAL_Delay() — all tasks are scheduled using HAL_GetTick() to maintain responsiveness. Safety First — EXTI0 (NVIC Priority 0) immediately preempts normal execution and activates the emergency path.
-
- Shared State Management — brake_emergency_flag and brake_emergency_active coordinate communication between the ISR and the main loop.
-
- Stable ADC Sampling — 480-cycle sampling improves TPMS stability in Proteus simulations. Dedicated UART Buffers — separate transmit buffers are used for normal and emergency telemetry frames.
-
- Interrupt-Driven Communication — HAL_UART_Transmit_IT() allows telemetry transmission without blocking ADC monitoring.

| Simulation Result & Validation

PROTEUS + VIRTUAL TERMINAL — THREE PHASES RUNNING TOGETHER

```
Tire Pressure: 3010  
Tire Pressure: 2985  
Tire Pressure: 1980  
Tire Pressure: 1745  
Tire Pressure: 1745 EMERGENCY  
Tire Pressure: 1730 EMERGENCY  
Tire Pressure: 1720 EMERGENCY
```

VALIDATED BEHAVIOUR

The moment the collision button is pressed in Proteus, the emergency path is activated: the red LED turns ON, the brake relay engages, and subsequent UART frames include the EMERGENCY tag.

ADC & UART Period

HAL_GetTick()-based scheduling for TPMS monitoring and telemetry transmission.

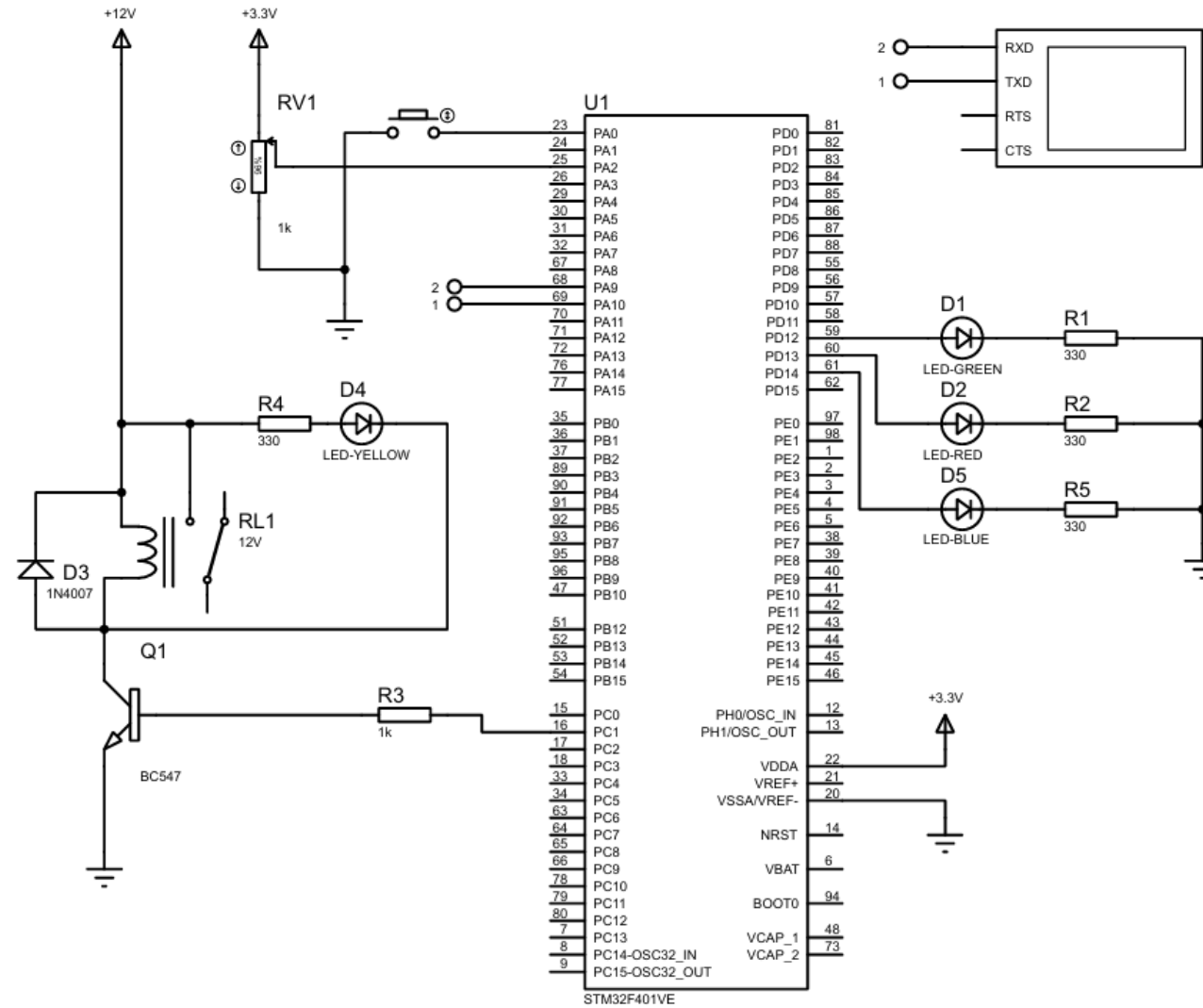
UART Link

Reliable delivery of both normal and emergency telemetry frames using HAL_UART_Transmit_IT().

Hardware Interrupt

Immediate transition from NORMAL mode to EMERGENCY mode upon a falling edge on PA0.

| Proteus



Safety node: ready for integration with the master.

END OF NODE 1

Next presenter: Node 2 — Environment & Light

Node2: Environment & Light

PRESENTER: PARINAZ KARI

Environmental Node for Continuous Lighting Control, Environmental Sensing, and Non-Blocking Telemetry.

| Node 2 — Mission & Responsibilities

WHAT THIS NODE MUST DO INSIDE THE SMART CAR

Manual & Auto Actuation

- Granular PWM headlight control.
- Hardware-driven dimming via physical inputs.

Ambient Light Monitoring

- Real-time tunnel detection using an LDR sensor
- Automatic PWM override

Thermal Monitoring

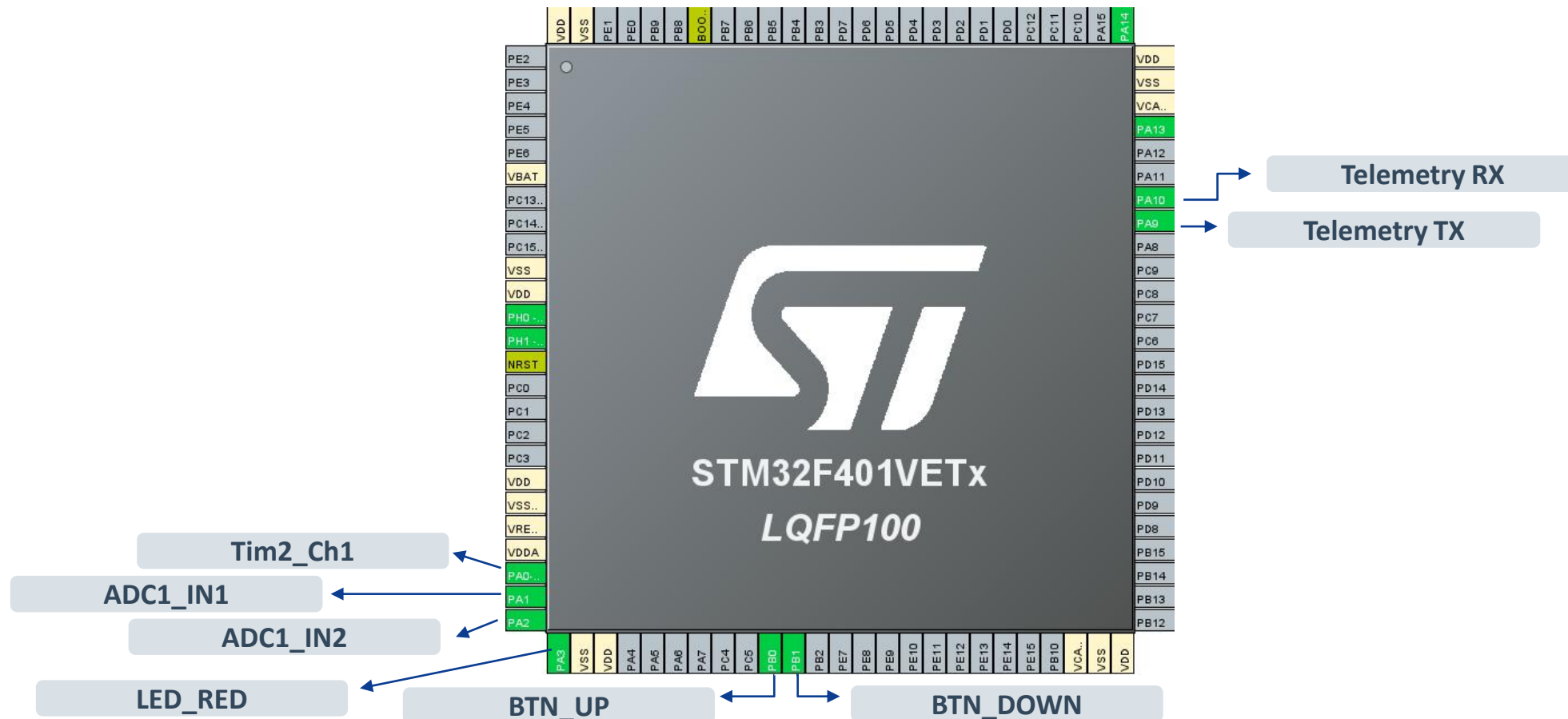
- Precision temperature sampling via LM35
- Dedicated data stream for the telemetry dashboard

Interrupt-Driven Telemetry

- Non-blocking UART transmission
- Continuous state reporting to the Master Node..

| STM32CubeMX Configuration

PERIPHERAL SETUP, PIN MAPPING, AND SYSTEM INITIALIZATION



| Three Development Phases

EACH PHASE ADDED ONE LAYER TO THE SAME FIRMWARE

Phase1

PWM Dimming Control

PWM output on a timer channel simulates manual brightness adjustment for LEDs/lights.

Duty cycle is controlled by potentiometer (manual input) to demonstrate variable intensity control.

PWM frequency and resolution are configured via timer hardware (HAL_TIM_PWM_*functions)

phase2

Automatic Lighting & Temperature Sensing

LDR sensor (light-dependent resistor) on ADC input for ambient light detection → automatically controls headlights based on brightness level.

LM35 temperature sensor on ADC input reads ambient temperature for monitoring or thermal management.

Both sensors are read via ADC and processed in the main loop with appropriate thresholds.

phase3

UART Telemetry

USART1 @ 9600 8N1 on PA9 transmits framed strings to the master, using HAL_UART_Transmit_IT. Telemetry data includes sensor readings and system status.

Node 2

Environment & Light

Timer and PWM- Simulate controlling the light intensity of lights by changing the PWM handle.

PHASE 1

Familiarity with the Timer unit in ARM Cortex-M4 architecture microcontrollers, PWM output configuration (pulse width modulation), setting the system clock frequency to the speed speed (84MHz), input management buttons with internal pull-up resistors

Configuration in STM32CubeMX software

1. Clock Configuration

Clock source: HSI (16 MHz internal)

PLL configured to achieve HCLK = 84 MHz

2. Timer 2 (TIM2) – PWM Output

Clock: Internal Clock

Channel 1: PWM Generation on PA0

Output frequency: 1 kHz

Prescaler = 83, ARR = 999

3. Button Inputs (GPIO)

PB0 = Increment button (BTN_UP)

PB1 = Decrement button (BTN_DOWN)

Both with internal Pull-up (High = idle, Low = pressed)

|PWM Generation & Button Interface @84MHz

WITH PRESCALER=83 AND ARR=999, THE OUTPUT PWM FREQUENCY IS EXACTLY 1 KHZ (1 MS PERIOD), WHICH IS STANDARD FOR MOTOR AND FADING CONTROL APPLICATIONS

$$\text{PWM Frequency} = \frac{\text{Timer Clock Channel}}{\text{ARR}+1} = \frac{1,000,000}{999+1} = 1,000 \text{ Hz} = 1\text{KHz}$$

$$\text{Timer Clock Channel} = \frac{\text{HCLK}}{\text{Prescaler}+1} = \frac{84,000,000}{83+1} = 1,000,000 \text{ Hz} = 1\text{MHz}$$

Control with buttons (in a while loop):

Debounce Time: 200ms

Pull-Up: Internal (Active LOW logic)

Range: 0 to 999 (0% to 100% duty cycle)

```
if ((HAL_GetTick() - last_button_tick) > 200)
{
    if (HAL_GPIO_ReadPin(BTN_UP_GPIO_Port, BTN_UP_Pin) == GPIO_PIN_RESET)
    {
        if (pwm_value < 999) pwm_value += 50;
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pwm_value);
        last_button_tick = HAL_GetTick();
    }
    else if (HAL_GPIO_ReadPin(BTN_DOWN_GPIO_Port, BTN_DOWN_Pin) == GPIO_PIN_RESET)
    {
        if (pwm_value > 0) pwm_value -= 50;
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pwm_value);
        last_button_tick = HAL_GetTick();
    }
}
```

| Challenge & Solution

ISSUE	SOLUTION
Proteus real-time graphic limitations at high clock speeds	Used software delay loop (volatile for) instead of HAL_Delay()
Blocking behavior from system delays	Small step size (50 units) ensured smooth transitions
Result:	Stable system performance with no blocking

SOLUTION

Smooth, continuous PWM fading effect achieved using 50-unit steps and software delay loop, eliminating blocking issues in Proteus simulation

Smart Environmental Control System

PHASE 2

Learn ADC configuration and multiplexed reading of multiple channels

Implement non-blocking architecture using HAL_GetTick() for task scheduling

Apply hysteresis-based logic for fan and lighting control

| ADC1 Configuration

CUBEMX SETTINGS FOR ADC1_IN2 ON PA2 FOR LM5 AND IN1 FOR LDR

PARAMETER	VALUE
Channel 1 (IN1)	PA1 → LDR (Light Sensor)
Channel 2 (IN2)	PA2 → LM35 (Temperature Sensor)
Scan Conversion Mode	ENABLED(multi-channel scanning)
Continuous Conversion Mode	ENABLED (Continuous operation)
Resolution	12-bit (0–4095)
EOC Selection	EOC flag at end of single channel (Critical for Proteus stability)

OUTPUT PIN SETUP

PA3 -> Digital Output -> RED_LED

LM35 THEORY & FORMULA

10 mV per °C output voltage

Reference Voltage: 3.3V

ADC Resolution: 12-bit (0–4095)

TEMPERATURE FORMULA:

$\text{ADC Value} = (\text{Vout} \times 4095) / 3.3\text{V}$

$\text{Vout} = \text{Temperature} \times 0.01\text{V}$

$\text{Result: ADC} \times 330 / 4095$

|Temperature Monitoring & Control - Automatic Light Control

LED WARNING LOGIC & AUTO MODE LOGIC

LED Warning Logic:

- $\text{Temp} > 40^{\circ}\text{C} \rightarrow \text{RED LED ON (Overheat Warning)}$
- $\text{Temp} < 37^{\circ}\text{C} \rightarrow \text{RED LED OFF (Normal Temperature)}$
- $37^{\circ}\text{C} \leq \text{Temp} \leq 40^{\circ}\text{C} \rightarrow \text{Maintain previous state}$

Auto Mode Logic ($\text{auto_light_mode} = 1$):

- $\text{ADC Value} > 2500$ (Dark environment) $\rightarrow$ Lights ON at 100%
- $\text{ADC Value} < 2000$ (Bright environment) $\rightarrow$ Switch to Auto Mode
- Manual Override:
- Pressing BTN_UP or $\text{BTN_DOWN} \rightarrow \text{auto_light_mode} = 0$ Auto mode resumes when light level drops below 2000

|LED Warning Logic & Auto Mode (in a while loop):

```
temperature = (adc_values[0] * 330) / 4095;
    if (temperature > 40)
        HAL_GPIO_WritePin(RED_LED_GPIO_Port, RED_LED_Pin, GPIO_PIN_SET);
    else if (temperature < 37)
        HAL_GPIO_WritePin(RED_LED_GPIO_Port, RED_LED_Pin, GPIO_PIN_RESET);
}
if (adc_values[1] > 2500 && auto_light_mode == 1) {
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 999); // روشنایی کامل
    pwm_value = 1000;
} else if (adc_values[1] < 2000) {
    auto_light_mode = 1; // بازگشت به حالت خودکار
}
```

Online Telemetry System

PHASE 3

Real-time transmission of system status to a monitoring station via UART.

|Sending periodically (in a while loop):

```
if (HAL_GetTick() - last_tick >= 500)
{

//ADC codes ...

char led_status[4];
    if (HAL_GPIO_ReadPin(RED_LED_GPIO_Port, RED_LED_Pin) == GPIO_PIN_SET) {
        sprintf(led_status, "ON");
    } else {
        sprintf(led_status, "OFF");
    }
    sprintf(tx_buffer, "Temp:%uC | Warn LED:%s | Lights PWM:%d\r\n",
        (unsigned int)temperature, led_status, pwm_value);
    HAL_UART_Transmit_IT(&huart1, (uint8_t*)tx_buffer, strlen(tx_buffer));
}
```

| ADC1 Configuration

CUBEMX SETTINGS FOR ADC1_IN2 ON PA2 FOR LM5 AND IN1 FOR LDR

PARAMETER	VALUE
USART Instance	USART1
Baud Rate	9600 bps
Data Bits	8 bits
Stop Bits	1 bit
Mode	TX and RX

TRANSMISSION INTERVAL

Every 500 ms

DATA FORMAT:

Temp:XXC | Warn LED:XXX | Lights PWM:XXX

EXAMPLE OUTPUT:

Temp:25C | Warn LED:OFF | Lights PWM:500
Temp:42C | Warn LED:ON | Lights PWM:750
Temp:23C | Warn LED:OFF | Lights PWM:999

|Challenge & Solution

Challenge

- Maintaining non-blocking architecture integrity during UART transmission
- Polling method (HAL_UART_Transmit) would block the CPU for several milliseconds while transmitting each character
- Result: Missing button edge detections on PB0 (UP) and PB1 (DOWN) → Loss of manual control responsiveness

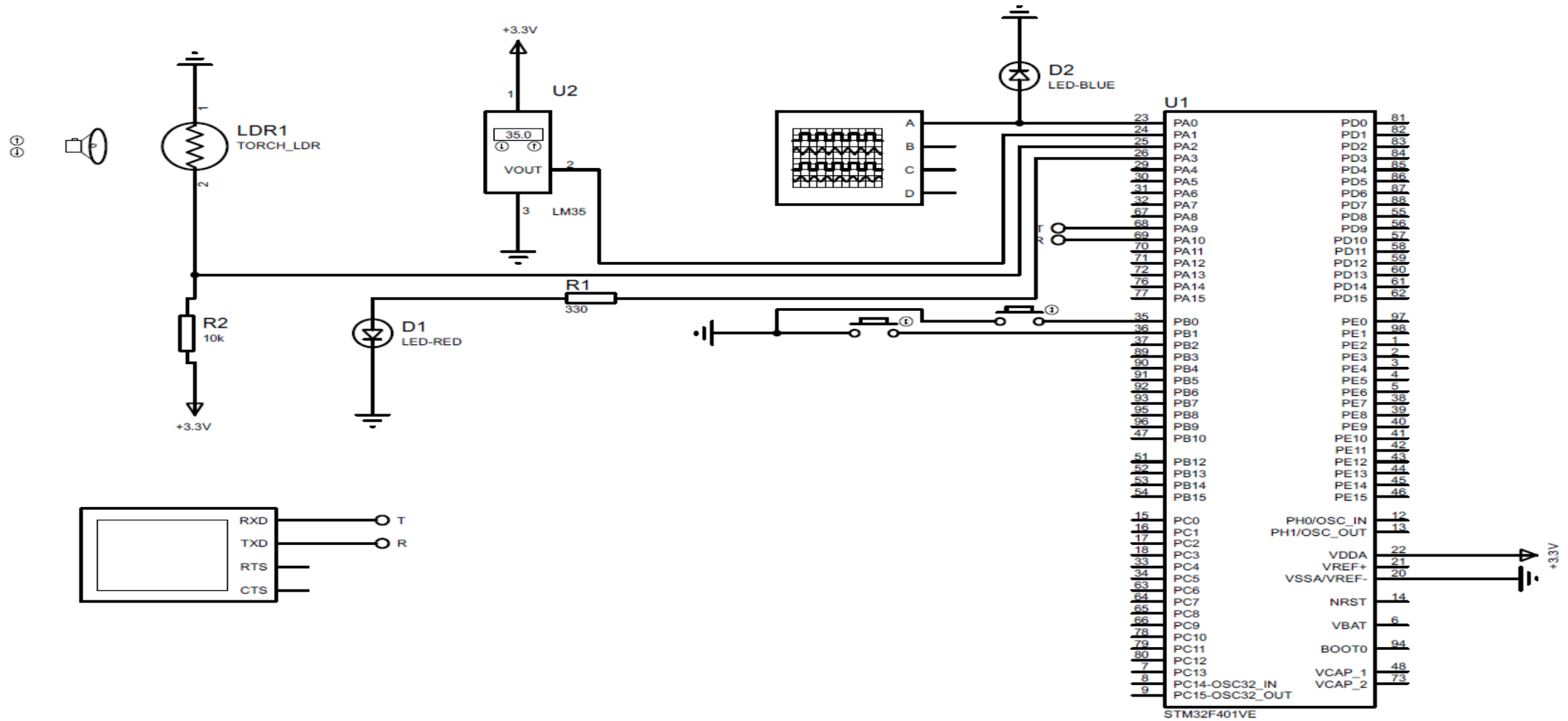
Solution

- Interrupt-driven transmission (HAL_UART_Transmit_IT()) implemented
- CPU only delivers buffer pointer to the hardware UART peripheral
- Core immediately returns to button monitoring tasks
- No blocking — continuous button scanning maintained

| Key Takeaways from

- The system implements a complete non-blocking architecture using `HAL_GetTick()` for task scheduling across all three phases.
- Phase 1: PWM-based manual light control with 50-unit fading steps, debounced buttons (200ms), and real-time oscilloscope observation.
- Phase 2: Smart environmental control with multi-channel ADC (LDR + LM35), hysteresis-based logic, and automatic/manual mode switching.
- Phase 3: Online telemetry with interrupt-driven UART transmission (9600 baud) and periodic status reporting every 500ms.
- All phases share a common design philosophy: NO `HAL_Delay()`, NO blocking operations, and ALL tasks scheduled using `HAL_GetTick()` for maximum system responsiveness.

Proteus



Environment & Light node: ready for integration with the master.

END OF NODE 2

Next presenter: Node 3 — Vehicle & Battery

Node3 : Vehicle & Battery

PRESENTER: MAHDI TAHOUNI

The vehicle status node monitors critical vehicle parameters, including battery voltage, motor speed, and operational statistics. It performs ADC-based sensor acquisition, processes real-time measurements, calculates vehicle state information, and transmits structured telemetry packets to the master node through UART communication using an interrupt-driven architecture.

| Node 3 — Mission & Responsibilities

WHAT THIS NODE MUST DO INSIDE THE SMART CAR

Accelerator Input & Speed Simulation

- Receiving accelerator pedal input through ADC
- Simulating vehicle speed based on pedal value

Motor PWM Control

- Generating PWM signal for motor control
- Controlling vehicle behavior using timer-based PWM

Battery Monitoring & Calculation

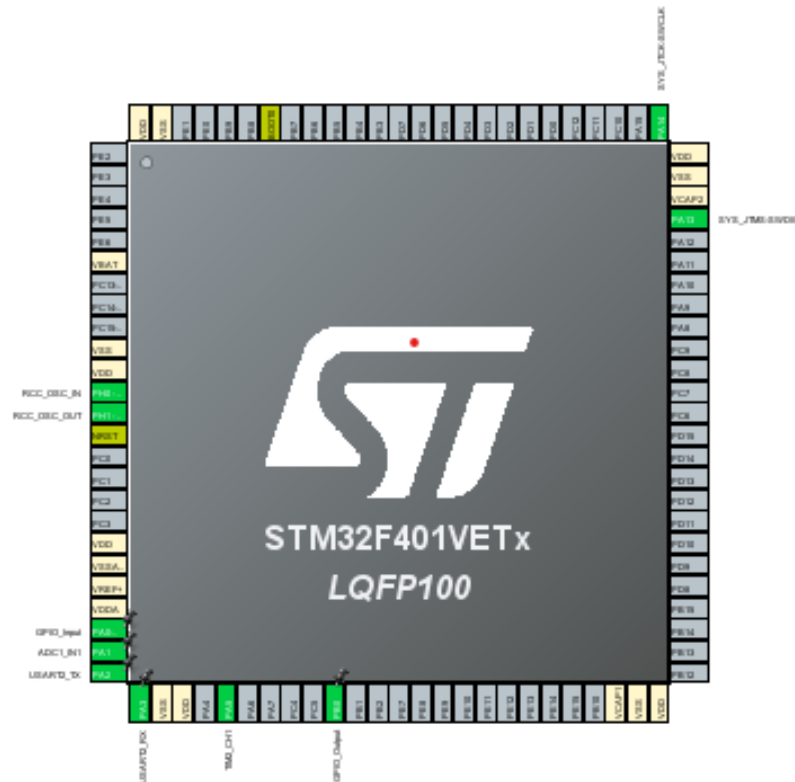
- Monitoring battery status from ADC measurement
- Calculating battery percentage estimation

UART Telemetry

- Sending speed and battery information to the Central Controller
- Providing real-time vehicle status reporting

| STM32CubeMX Configuration

PERIPHERAL SETUP, PIN MAPPING, AND SYSTEM INITIALIZATION



Default State	Role in Circuit	Pin Type	Pin Name
Analog	Gas pedal & battery voltage input	ADC1_IN1	PA1
AF7	Transmit data to central core	USART2_TX	PA2
AF7	Receive data	USART2_RX	PA3
Low	PWM pre-driver	TIM2_CH1	PA5
Low	Speed display LED	GPIO_Output	PB0

| Three Development Phases

EACH PHASE ADDED ONE LAYER TO THE SAME FIRMWARE

Phase1

The main objective was to read the accelerator pedal position using an Analog-to-Digital Converter (ADC) and convert it into the vehicle's speed. Additionally, to provide a visual representation of speed changes, an LED was used as an indicator, with its blinking rate varying proportionally to the vehicle's speed. This phase served as the fundamental part of the project and formed the basis for the implementation of the subsequent phases.

Phase2

Although an actual DC motor was not connected in the Proteus simulation, a PWM signal was generated on the designated output pin so that it can be directly connected to a motor driver in future implementations. This approach makes the motor node closer to a real motor control system rather than a simple simulation. **Key Features:**
PWM signal generation using **TIM2**.
Conversion of the calculated speed percentage into the corresponding **PWM duty cycle**.
Preparation of a suitable PWM output for future **DC**

Phase3

Objectives:

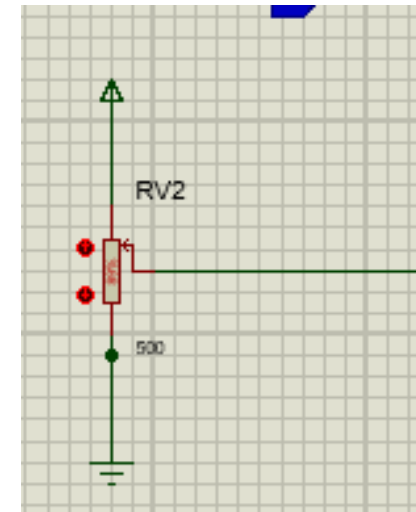
Transmit motor-related information to the **Central Controller**.
Construct a structured **Data Packet** for communication.
Calculate and append a **Checksum** to enhance data integrity.
Transmit data through the **UART** interface.
Support debugging and monitoring using the **Proteus Virtual Terminal**

Node 3

Vehicle & Battery

| Phase 1: Engine Speed Simulation

In this phase, a potentiometer is used to simulate the vehicle's accelerator pedal.
By rotating the potentiometer:
Potentiometer Change -> Analog Voltage Change ->
ADC1 -> Digital Value 0 to 4095 -> Speed Calculation
-> LED Blinking Frequency Change
As the accelerator pedal is increased, the LED
blinking frequency increases.
The LED therefore provides a visual representation
of the simulated vehicle speed.

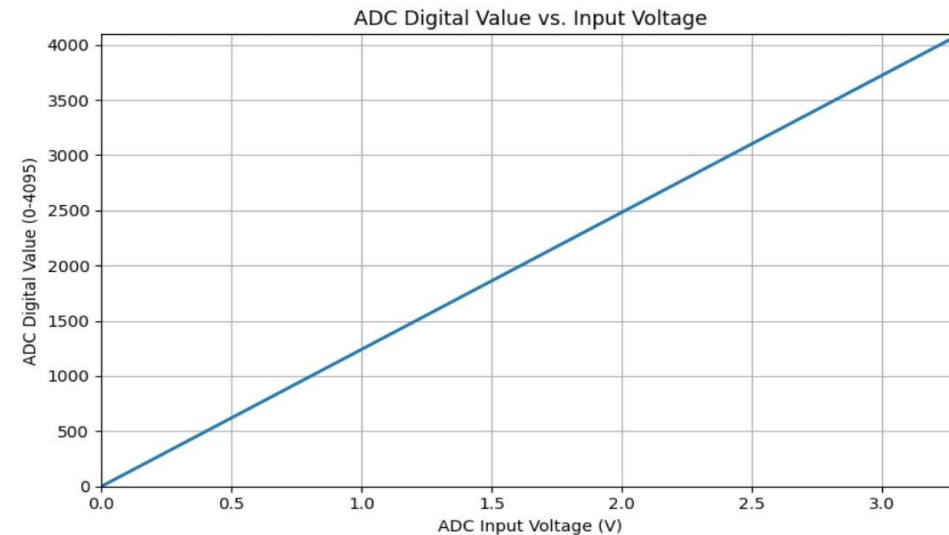


|ADC and Input Voltage Relationship

THE STM32 12-BIT ADC GENERATES A DIGITAL VALUE BETWEEN 0 AND 4095.
ASSUMING A REFERENCE VOLTAGE OF 3.3V

The relationship between ADC input voltage and the digital ADC value is linear.

$$ADC = \frac{V_{in}}{3.3} \times 4095$$



| Battery Percentage Calculation

FOR DISPLAYING THE BATTERY STATUS ON THE LCD, THE ADC VALUE IS CONVERTED INTO A PERCENTAGE

The battery percentage is assumed to have a linear relationship with the ADC measurement within the specified operating range.

$$Battery\% = \frac{Speed}{1.8}$$

$$Battery\% = \frac{ADC_{bat} \times 100}{4095}$$

$$ADC_{gas} = ADC_{bat}$$

$$Speed = \frac{ADC_{gas} \times 180}{4095}$$

| Phase 2: PWM Motor Control

Accelerator -> ADC -> Motor Speed % -> PWM Duty Cycle -
> TIM2_CH1 -> Motor Control

The main purpose of this phase is to prepare the system for connection to a real motor or Motor Driver.

The system:

Reads the ADC value.

Calculates the motor speed.

Updates the PWM duty cycle according to the speed.

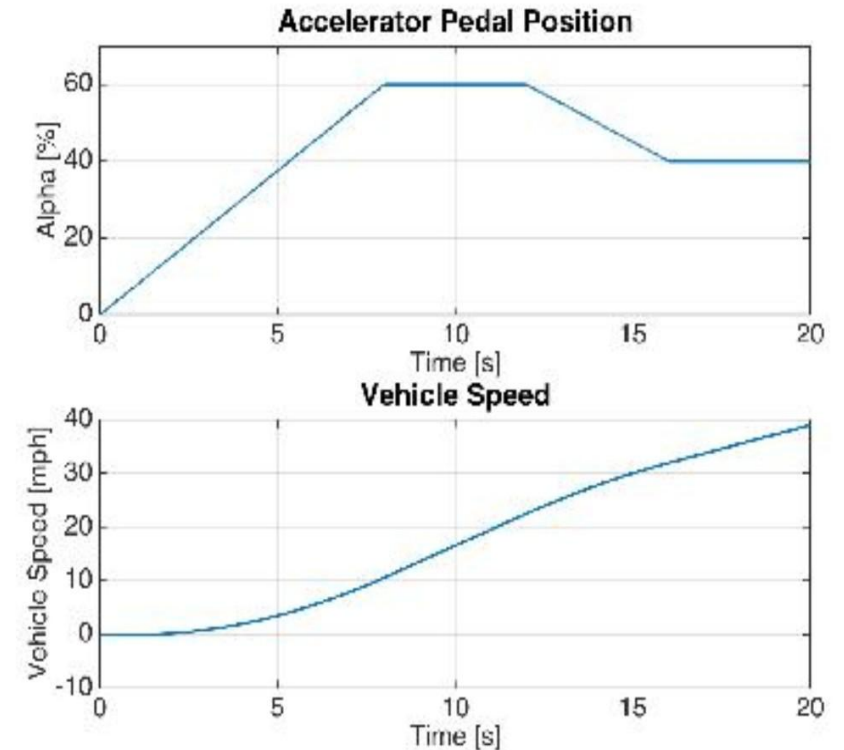
Generates the PWM output using TIM2.

| Accelerator Pedal and Vehicle Speed Relationship

THE ACCELERATOR PEDAL INPUT IS MEASURED USING THE ADC.
IN THE CURRENT IMPLEMENTATION

Accelerator 0% → Speed = 0 km/h
Accelerator 50% → Speed ≈ 90 km/h
Accelerator 100% → Speed = 180 km/h

This provides a simple linear relationship between accelerator pedal position and simulated vehicle speed. However, it should be noted that the speed assigned to each accelerator pedal position represents only the **average speed** over the corresponding time interval. In reality, the speed changes over time according to the graph shown opposite.



| Phase 3: Communication & Telemetry

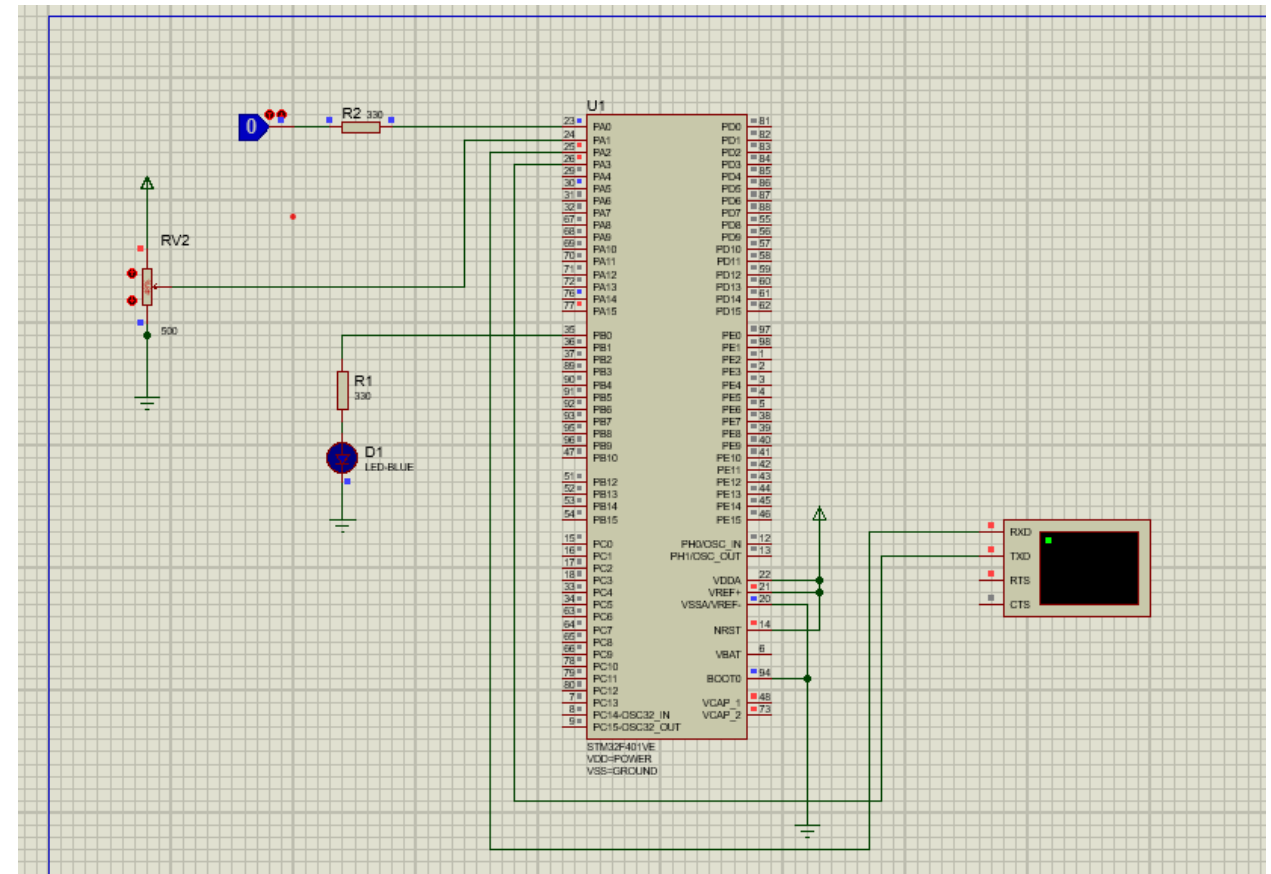
The main transmitted data includes:

Vehicle speed

Battery percentage

For initial communication testing, a Virtual Terminal in Proteus is used.

The **Logic State** component was used only during the initial stages of **Phase 1** and does not play any significant role in the subsequent phases.

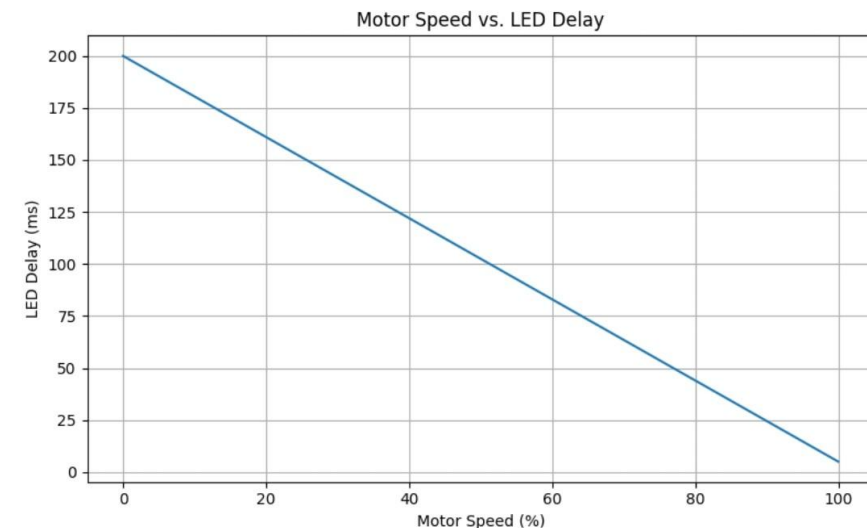


| LED-Based Speed Simulation

TO PROVIDE A VISUAL REPRESENTATION OF THE VEHICLE SPEED, THE LED IS PERIODICALLY TURNED ON AND OFF. THE DELAY IS INVERSELY RELATED TO THE SIMULATED SPEED

From a human visual perspective, the increasing LED blinking frequency represents an increase in the simulated vehicle speed. The purpose of this method is to provide a simple visual simulation of speed behavior without requiring a physical speed sensor or a real rotating wheel.

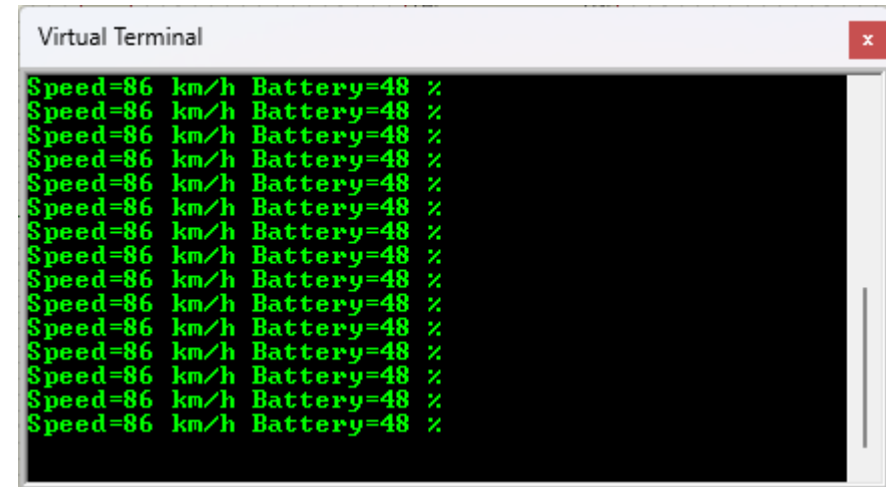
$$[\text{Delay} = 200 - (\text{Speed} \times 1.95)]$$



|Virtual Terminal in Proteus

THE VIRTUAL TERMINAL IS USED TO TEST AND VERIFY SERIAL COMMUNICATION.

The Virtual Terminal settings must match the UART configuration:
Baud Rate = 9600 Data Bits = 8 Parity = None Stop Bits = 1
During the debugging stage, messages such as:
Speed=90 km/h Battery=75 %
can be displayed to verify the UART communication.



Vehicle & Battery: ready for integration with the master.

END OF NODE 3

Next presenter: Node 4 — **Master Core & Display Node**

Node 4: Master Core & Display Node

PRESENTER: SALAR GHASEMI

The master node aggregates telemetry from all subsystems, manages a real-time LCD interface, synchronizes UART communication through a multiplexer, and streams consolidated vehicle data to the dashboard using a fully interrupt-driven architecture.

| Node 4 - Master Core & UI Aggregator

CENTRAL COMMUNICATION HUB, LCD INTERFACE & MULTI-NODE ORCHESTRATOR

Core Responsibilities

- Central Communication Hub: Acts as the UART Master, receiving telemetry from all 4 slave nodes simultaneously.
- UI Aggregator: Displays real-time data from Safety, Environment, Vehicle, and AI subsystems on a character/graphical LCD.
- Menu Navigation Controller: Manages multi-tiered screen menus via hardware pushbuttons (EXTI/pollled GPIO).
- Web Gateway: Serializes all node states into JSON and pipes telemetry to the Node.js WebSocket dashboard via COMPIM.
- Emergency Alert Manager: Hijacks the display to show critical safety warnings (brake triggers, tire pressure drops) with highest priority.

Peripheral Configuration (STM32CubeMX)

- USART1: Safety / Vehicle Data Reception @ 9600 baud
- USART2: Environment / AI Data Reception @ 9600 baud
- USART6: JSON Telemetry Transmission to Dashboard
- GPIO pins for LCD and menu navigation buttons
- Hardware MUX control via GPIOC Pin 3

MCU PLATFORM

STM32F401VET6 — ARM Cortex-M4F @ 84 MHz

Display Module

16×2 Character LCD (HD44780)

Communication

3× USART + Hardware UART Multiplexing @ 9600 bps

Input Interface

2× Pushbuttons (Next / Previous Menu)

| Phase 1–2: Peripheral Init & Menu

BUILDING THE FOUNDATION — FROM LCD DRIVER TO REAL-TIME MONITORING INTERFACE

Phase 1 — Digital Baseline

- Configured GPIO and USART peripherals via STM32CubeMX for STM32F401VET6.
- Implemented a HD44780-compatible 16×2 LCD driver using HAL GPIO bit-banging.
- Verified LCD initialization timing and command sequencing.
- Displayed startup messages to validate hardware functionality.

Phase 2 — User Interface Development

- Designed a four-page menu system for real-time subsystem monitoring.
- Implemented two hardware pushbuttons (Next/Previous) using polled GPIO inputs.
- Added debounce handling and clean LCD refresh mechanisms.
- Dynamically updates menu content with live telemetry received over UART.

Menu Pages

Speed & Battery Menu

Node 3 Data — Live speed (km/h), battery voltage (%), odometer count

Safety & TPMS Menu

Node 1 Data — Tire pressure, collision sensor state, brake status

Environment & Climate Menu

Node 2 Data — Temperature (°C), ambient light (LDR), headlight PWM %

AI Obstacle Alerts

Node 5 Data — High-priority obstacle distance, path clear status, TinyML inference result

IMPLEMENTATION NOTES

Menu state is maintained using a `current_menu` variable. The main loop updates the active LCD screen based on received UART telemetry and user navigation input.

| Phase 3: Real-Time Multi-Node Data Aggregation

UART MASTER CORE — RECEIVING, PARSING & SYNCHRONIZING TELEMETRY

Networking Role

- Acts as the central UART aggregation node for the Smart Car system.
- Receives asynchronous telemetry through USART1 and USART2.
- Utilizes a hardware multiplexer to synchronize communication between multiple subsystems.
- Messages are ASCII-based and terminated with CR/LF for reliable packet detection.

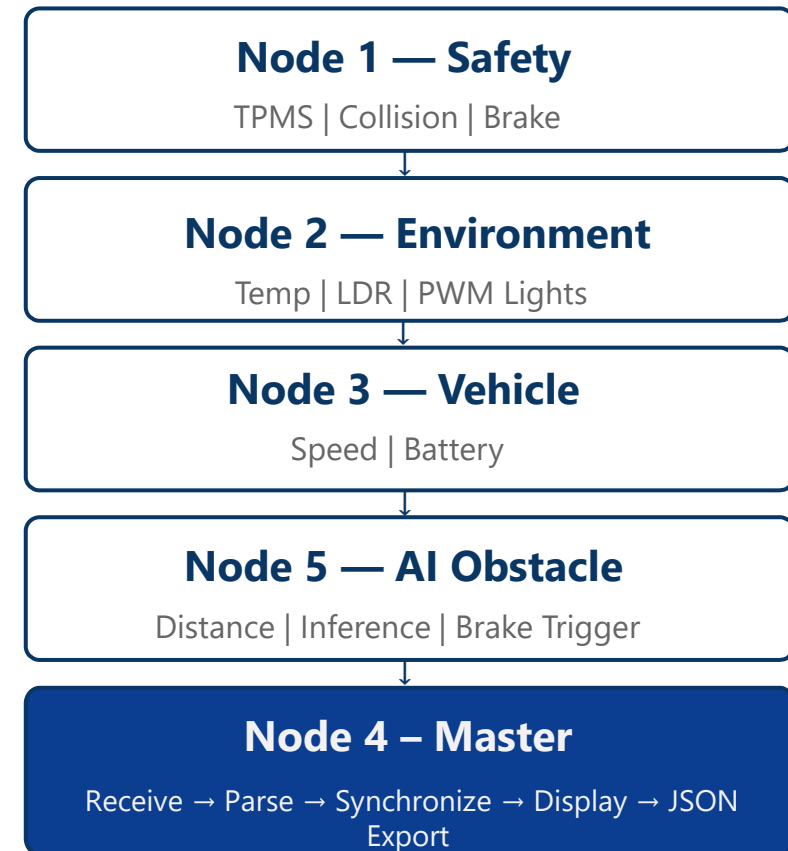
Technical Execution

- Dual interrupt-driven UART receivers using HAL_UART_Receive_IT().
- Dedicated receive buffers for independent data streams.
- Uses strstr() and sscanf() to identify and parse telemetry packets.
- Parsed values are stored in global variables for immediate LCD updates and dashboard transmission.

EXAMPLE PARSING FLOW

Incoming: "Speed=45 Battery=78" → strstr("Speed=") → atoi() → car_speed = 45 → LCD updated during the next main-loop iteration

Data Flow Architecture



| Dashboard Integration & Telemetry Streaming

REAL-TIME MONITORING, JSON EXPORT & WEB SYNCHRONIZATION

Dashboard Integration

- Aggregates telemetry from all subsystems into a unified JSON payload.
- Transmits telemetry through USART6 to the Proteus COMPIM interface.
- Node.js parses incoming serial data and broadcasts updates via WebSocket.
- Browser dashboard presents live Safety, Environment, Vehicle, and AI status.

Real-Time Status Handling

- Displays safety status (SAFE/DANGER) on the LCD.
- Updates obstacle detection state (Clear/Obstacle) in real time.
- Provides immediate visibility of tire pressure, speed, battery, and temperature data.

Example JSON Packet

```
{  
  "safety":{"tire":32,"danger":0},  
  "env":{"temp":24,"pwm":80},  
  "status":{"speed":45,"bat":78},  
  "ai":{"obstacle":"Clear"}  
}
```

| Technical Challenges & Optimization

NODE 4 SPECIFIC HURDLES — UART SYNCHRONIZATION, MUX CONSTRAINTS & SIMULATION PERFORMANCE

Challenge 1: MUX Source Identification

- The hardware multiplexer is externally synchronized and not controlled by the STM32 master node. Consequently, Node 4 cannot explicitly determine the physical source of incoming telemetry after multiplexing.
- This trade-off was accepted because the current implementation is receive-only. The master node aggregates and displays telemetry without transmitting commands back to slave nodes.
- Future Enhancement: Introduce Node IDs or address-based packets for bidirectional communication support.

Challenge 2: UART Synchronization & Packet Integrity

Frequent MUX switching occasionally caused incomplete UART frames during Proteus simulation.

Solution

- Interrupt-driven UART reception (HAL_UART_Receive_IT())
- CR/LF packet synchronization
- UART error recovery (HAL_UART_ErrorCallback())
- Automatic buffer reset and receiver restart

Challenge 3: Proteus Simulation Performance

Simultaneous execution of multiple STM32 nodes, UART channels, LCD updates, and dashboard communication increased simulation overhead.

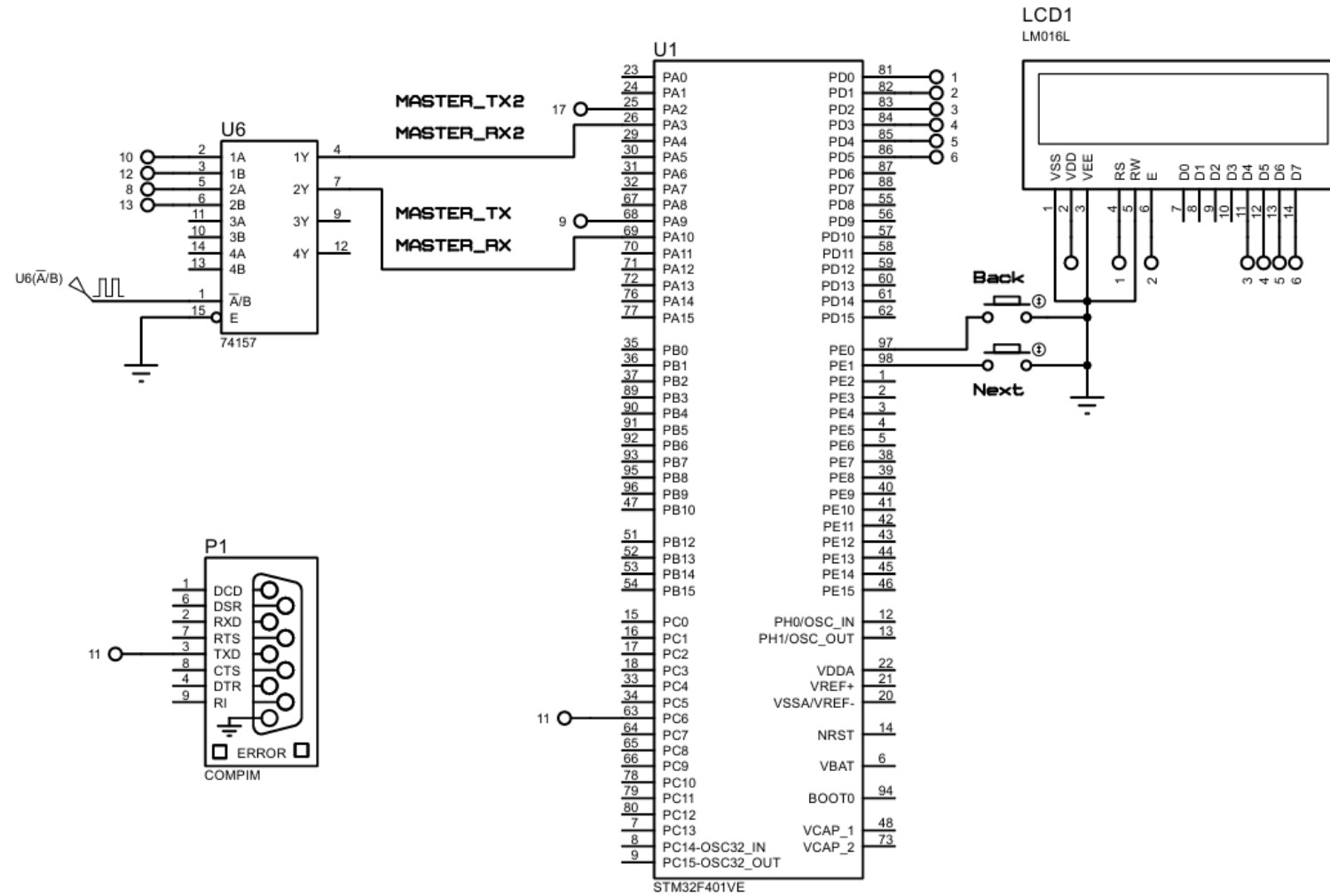
Optimizations

- Non-blocking communication architecture
- Fixed-size RX buffers
- Minimal ISR processing
- JSON telemetry streaming to the Node.js dashboard

Key Takeaway

Node 4 successfully operates as a lightweight telemetry aggregation hub, providing reliable multi-node monitoring through UART, LCD visualization, and real-time dashboard integration.

| Proteus



Master node: ready for AI-based decision integration.

END OF NODE 4

Next presenter: Node 5 — AI TinyOD-Brake

Node 5: Ai TinyOD-Brake

PRESENTER: SOBHAN NASIRI

TinyML-based obstacle detection and emergency braking system simulated on STM32F401VE using STM32Cube.AI, CMSIS-NN, and Proteus. ONNX model deployed on Arm Cortex-M4.

| Node 5 — Mission & Responsibilities

WHAT THIS NODE MUST DO INSIDE THE SMART CAR

Environmental Perception

- Captures a simulated 96×96 grayscale image, applies fixed-point preprocessing (no FPU), and runs an INT8 CNN inference to detect obstacles in real time.

Risk Assessment & TTC Calculation

- Uses simulated distance and speed to compute Time-to-Collision (TTC) with integer arithmetic; the TTC value determines the urgency of the response.

Finite-State Machine(FSM) Decision Logic

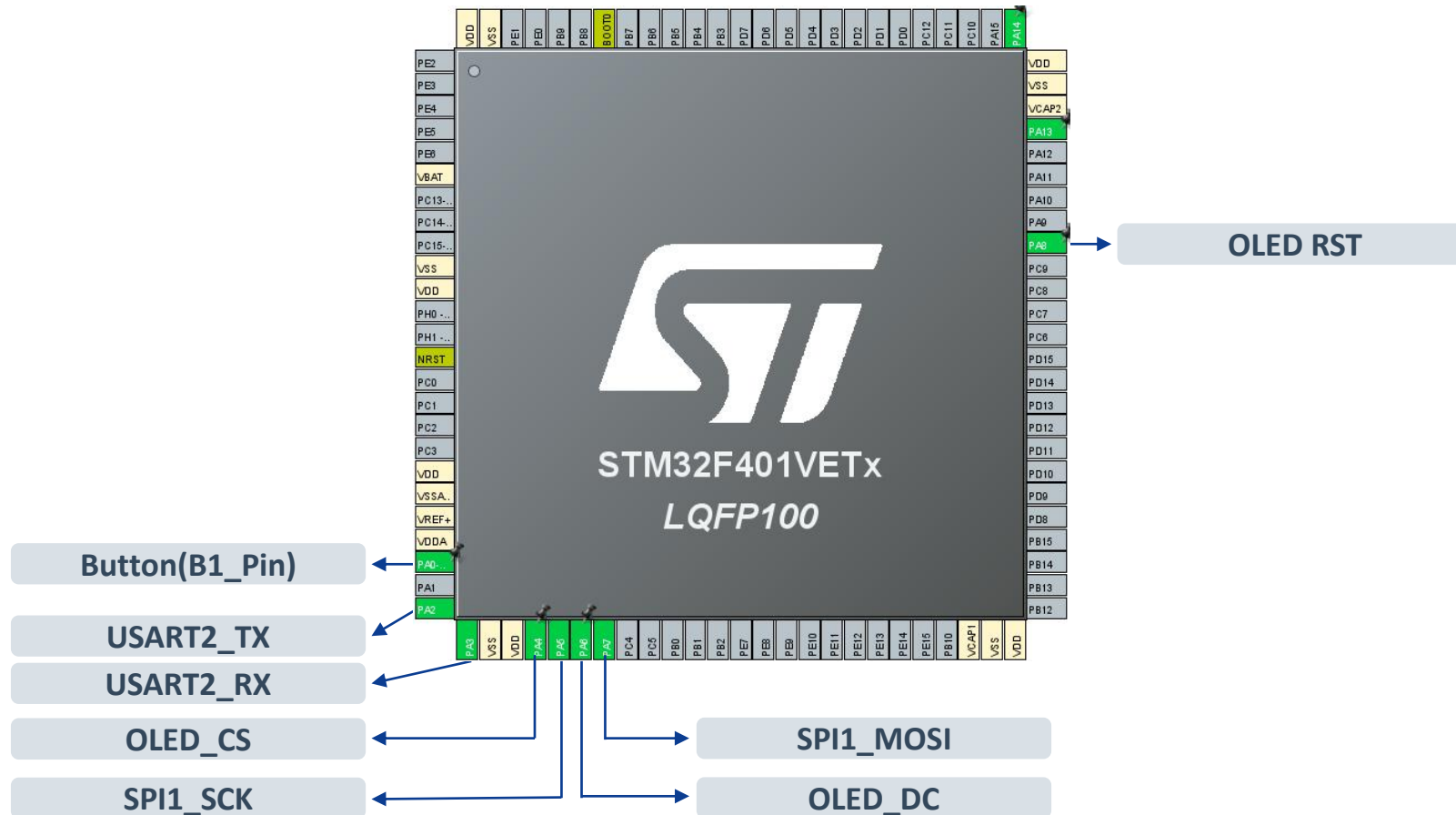
Implements four safety states (Normal, Warning, Brake, Safe Stop) and transitions based on obstacle probability and TTC thresholds to ensure controlled braking.

Graphical LCD Display

- Shows live system status, obstacle probability, TTC, and current state on a 240×320 ILI9341 LCD via SPI, updated at ~4 FPS for clear operator feedback.

| STM32CubeMX Configuration

PERIPHERAL SETUP, PIN MAPPING, AND SYSTEM INITIALIZATION



| Three Development Phases

EACH PHASE ADDED ONE LAYER TO THE SAME FIRMWARE

Phase1

Infrastructure, Data & Model Development

Environment Setup: STM32CubeMX, Keil MDK, Proteus simulator
Hardware Configuration: STM32F401VE (84 MHz, 96 KB RAM, 512 KB Flash)
Synthetic Dataset Generation: 96×96 grayscale images (noise + random obstacle squares)
Data Preprocessing: Fixed-point normalization (no FPU)
CNN Design: Lightweight depthwise-separable architecture (~16k params)
Training & Optimization: PyTorch → ONNX → (STM32Cube.AI)

phase2

TinyML Deployment & Decision Logic

AI Inference: CMSIS-NN accelerated inference on STM32 (bare-metal C)
Sensor Simulation: Dynamic frame generator (obstacle size/probability)
Risk Assessment: Time-to-Collision (TTC) calculation from simulated distance/speed
Finite State Machine (FSM): States – NORMAL, WARNING, BRAKE, SAFE_STOP
Graphical LCD (ILI9341): Displays state, obstacle probability, TTC, and live frame
UART Logging: Transmits inference latency, RAM usage, and state changes for debugging

phase3

Integration, Emergency Scenarios

System Integration: FSM output to actuator signals (GPIO toggles)
Team Coordination: Send state and speed data to other modules (team members 4 & 3)
Emergency Scenarios: TTC-based braking logic with <50ms reaction time
IoT (Optional): Transmit emergency braking status via UART / wireless link
Full Proteus Simulation: Virtual terminal, LCD, and cycle-accurate timing validation
Performance Metrics: ~74 KB RAM, ~25.8 FPS, 38.7ms inference latency

Node 5

Ai TinyOD-Brake

Infrastructure, Data & Model Development

PHASE 1

This phase includes the complete steps of building, training and using a very lightweight machine model for use in a Microcontroller Unit (MCU). The codes are written in a modular manner.

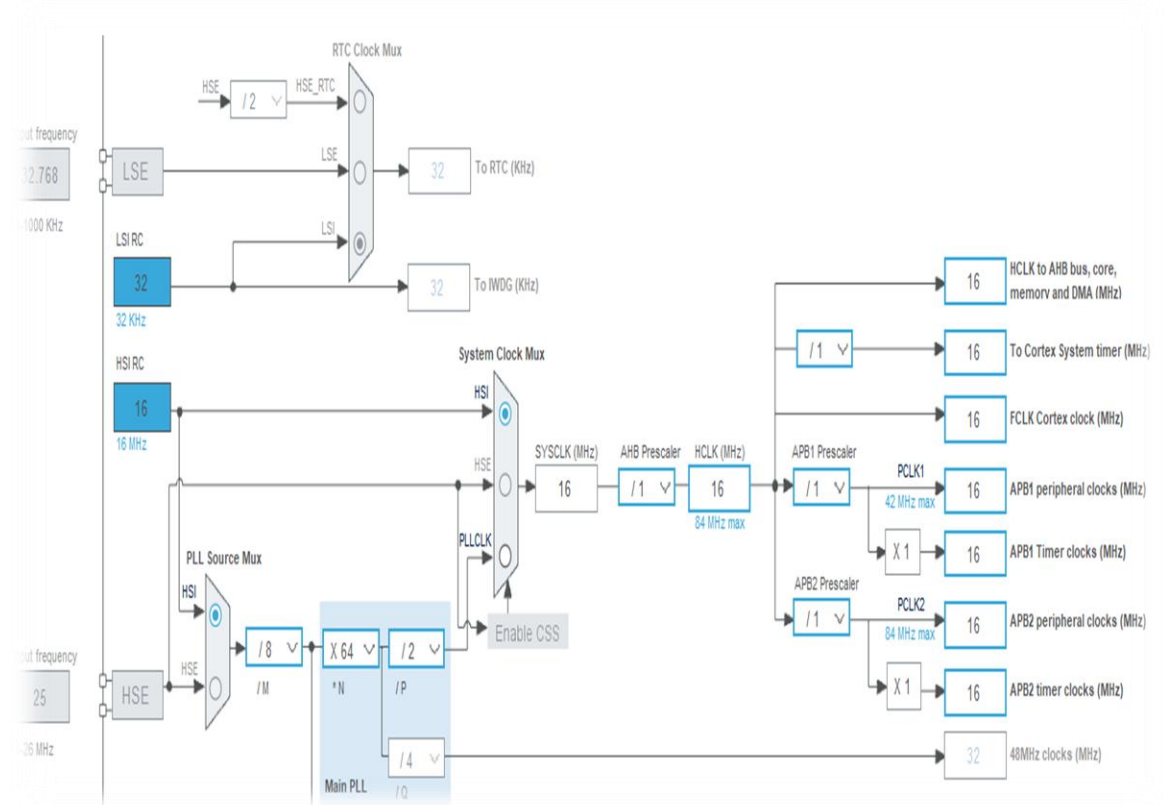
| Development Toolchain & Hardware Platform

Setting up the environment before writing a single line of AI code

Target MCU: STM32F401VET6, LQFP100 package, Arm Cortex-M4 core — selected for its balance of processing power and TinyML-class memory (96 KB RAM / 512 KB Flash). System

Clock: Configured via STM32CubeMX at 16 MHz (HSI-derived SYSCLK), propagated to AHB/APB1/APB2 buses at 16

MHz. Toolchain: STM32CubeMX for pin/clock configuration, Keil MDK-ARM (uVision 5) for firmware development, Proteus 8 Professional for full hardware-software co-simulation (no physical board used).



| Synthetic Dataset Generation

No real-world dataset available — so one was generated programmatically

Why synthetic: No camera or real obstacle dataset was available for this MCU-only pipeline, so `generate_data.py` procedurally builds the training set.

Class 0 — no_obstacle: Background noise only.

Class 1 — obstacle: Same background noise plus a randomly placed dark square simulating an obstacle.

Volume: 200 synthetic 96×96 grayscale images per class (400 total).



| Data Preprocessing & Loading Pipeline

Preparing images for a memory-constrained MCU

Custom Dataset Class: TinyObstacleDataset in data_loader.py reads all generated images via PIL.

Resize: All images normalized to a fixed 96×96 resolution to match the model's expected input.

Grayscale Conversion: Images converted to single-channel grayscale — cuts RAM footprint 3× versus RGB and speeds up inference.

File Name	Size	Type
__pycache__	14.0/0.4/17.0	File folder
dataset	14.0/0.3/16.0	File folder
best_tiny_model.pth	14.0/0.4/17.0	PTH File
data_loader.py	14.0/0.3/16.0	Python Source File
export_onnx.py	14.0/0.3/16.0	Python Source File
generate_data.py	14.0/0.3/16.0	Python Source File
model_F4.py	14.0/0.4/17.0	Python Source File
model_H7.py	14.0/0.3/16.0	Python Source File
obstacle_model.onnx	14.0/0.3/16.0	ONNX File
QN.py	14.0/0.3/16.0	Python Source File
test.py	14.0/0.4/17.0	Python Source File
train.py	14.0/0.3/16.0	Python Source File
train_F4.py	14.0/0.3/16.0	Python Source File

| CNN Architecture — TinyObstacleNetMCU

A custom, MCU-optimized convolutional network

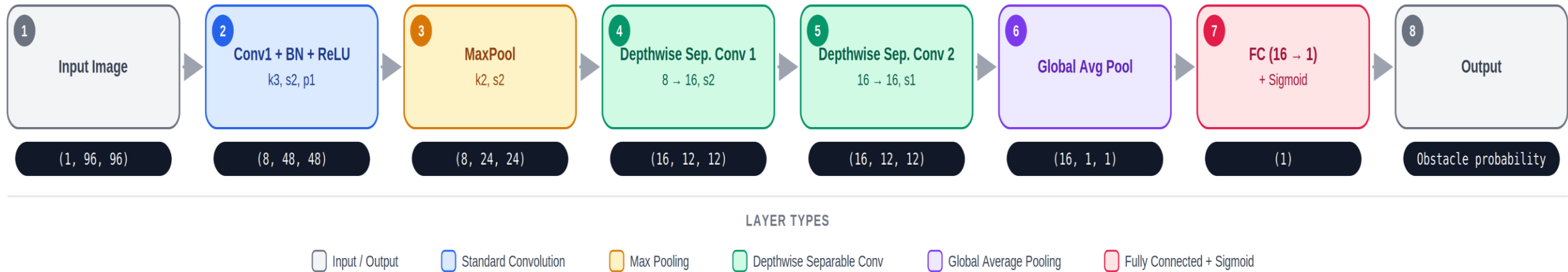
Design Goal: A binary classifier small enough to fit STM32F401VE's memory and compute budget.

Key Technique — Depthwise Separable Convolution (DSC): Splits a standard convolution into a depthwise pass + a 1×1 pointwise pass, drastically cutting parameter count and MACC operations versus a standard Conv2d.

Output Head: Global Average Pooling $\rightarrow$ Fully Connected ($16 \rightarrow 1$) $\rightarrow$ Sigmoid, producing an obstacle probability between 0 and 1.

TinyObstacleNetMCU — Layer-by-Layer Flow

Tracing a 96×96 grayscale input through the network



Design note: The number of filters in the first layer (8) is a manually chosen hyperparameter, not something derived from a formula — only the spatial output size of each layer follows the convolution/pooling formulas.

| Layer-by-Layer Output Dimensions

Tracing a 96×96 input through the network

Stage	Layer	Output Shape
Input	-	(1, 96, 96)
conv1	Conv2d(1→8, k3, s2, p1) + BN + ReLU	(8, 48, 48)
pool	MaxPool2d(k2, s2)	(8, 24, 24)
conv2	DepthwiseSepConv(8→16, s2)	(16, 12, 12)
conv3	DepthwiseSepConv(16→16, s1)	(16, 12, 12)
GAP	AdaptiveAvgPool2d(1,1)	(16, 1, 1)
FC	Linear(16→1) + Sigmoid	(1)

Use the table itself as the slide's main visual — no external image is needed here.

| Model Training & Export

From PyTorch weights to a hardware-portable format

Split: 80/20 random train/validation split.

Loss & Optimizer: Binary Cross-Entropy (BCE) loss with the Adam optimizer.

Schedule: 10 training epochs; the model checkpoint is saved whenever validation loss improves.

Result: Training accuracy reached 100% and validation accuracy reached 100% by epoch 8–10, with validation loss converging to 0.4497.

Export: The best checkpoint (best_tiny_model.pth) is exported to the ONNX (Open Neural Network Exchange) format via `export_onnx.py`, decoupling the model from PyTorch so it can be translated to C by hardware vendor tools.

```
Sobhan | torch_env | Ai_Model | main | ?1 -2 | 3.10.19 | 8.833s | python .\train.py
[*] Training on device: cuda
[*] Starting Training Loop...
Epoch [1/10] Train Loss: 0.6755, Train Acc: 60.62% | Val Loss: 0.7066, Val Acc: 40.00%
-> Best model saved with Val Loss: 0.7066
Epoch [2/10] Train Loss: 0.6498, Train Acc: 70.62% | Val Loss: 0.6935, Val Acc: 40.00%
-> Best model saved with Val Loss: 0.6935
Epoch [3/10] Train Loss: 0.6245, Train Acc: 75.62% | Val Loss: 0.6776, Val Acc: 66.25%
-> Best model saved with Val Loss: 0.6776
Epoch [4/10] Train Loss: 0.5945, Train Acc: 81.25% | Val Loss: 0.6550, Val Acc: 71.25%
-> Best model saved with Val Loss: 0.6550
Epoch [5/10] Train Loss: 0.5655, Train Acc: 85.94% | Val Loss: 0.6145, Val Acc: 96.25%
-> Best model saved with Val Loss: 0.6145
Epoch [6/10] Train Loss: 0.5329, Train Acc: 92.50% | Val Loss: 0.5655, Val Acc: 97.50%
-> Best model saved with Val Loss: 0.5655
Epoch [7/10] Train Loss: 0.5070, Train Acc: 93.75% | Val Loss: 0.5277, Val Acc: 97.50%
-> Best model saved with Val Loss: 0.5277
Epoch [8/10] Train Loss: 0.4730, Train Acc: 99.38% | Val Loss: 0.5027, Val Acc: 100.00%
-> Best model saved with Val Loss: 0.5027
Epoch [9/10] Train Loss: 0.4467, Train Acc: 99.38% | Val Loss: 0.4746, Val Acc: 100.00%
-> Best model saved with Val Loss: 0.4746
Epoch [10/10] Train Loss: 0.4207, Train Acc: 100.00% | Val Loss: 0.4497, Val Acc: 100.00%
-> Best model saved with Val Loss: 0.4497
[*] Training complete. Best model saved to best_tiny_model.pth
```

TinyML Deployment & Decision Logic Software development

PHASE 2

In the second phase, the written code is structured and various modules, including the sensor simulator, the AI Inference Engine, the Risk Calculator, and the Finite State Machine (FSM), are integrated together (Integration). This phase forms the core logic of the program.

And we will transfer the training model from the previous phase to a real hardware (microcontroller). This phase includes configuring the hardware, deploying the model using the manufacturer's development tools, setting up the display to show visual output, and setting up the serial port for debugging.

| Model Deployment via STM32Cube.AI

Translating ONNX into optimized embedded C

Import: The ONNX model is imported into X-CUBE-AI, which converts the Conv2D / Pooling / Dense graph into C code backed by CMSIS-NN kernels.

Memory Footprint: 40.22 KiB RAM (activations) + 3.74 KiB (library) $\approx$ 43.96 KiB total RAM; 23.19 KiB Flash for weights.

Compute Cost: $\approx$ 296,155 MACC (Multiply-Accumulate) operations per inference — confirmed by Cube.AI as feasible for this MCU.

Model manager			
Name	Used Ram	Used Flash	Complexity
ob_model	40.22 KiB	23.19 KiB	296155 MACC
Library	3.74 KiB	.00 B	
Total (1)	43.96 KiB	23.19 KiB	296155 MACC

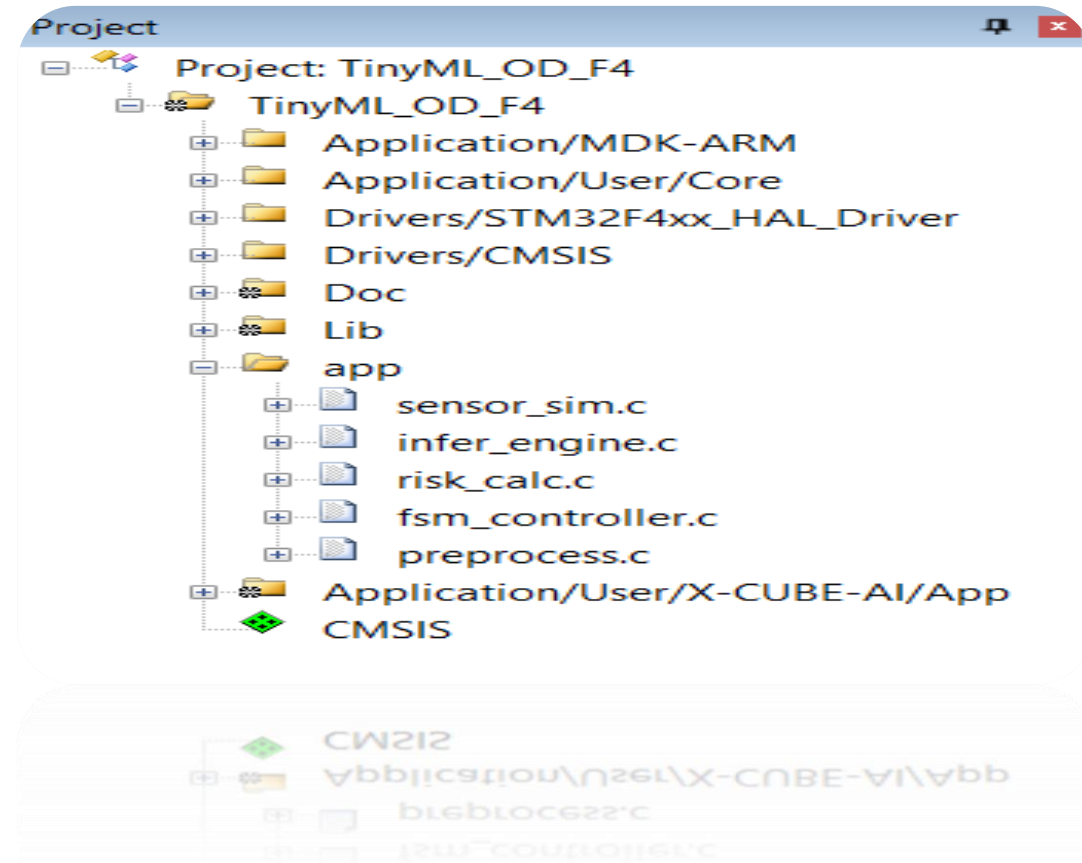
| Software Architecture & Modularization

Keeping main.c clean through modular design

Principle: Rather than crowding logic into main.c, functionality is split into a dedicated app/ folder.

Modules: sensor_sim.c (frame generation), infer_engine.c (AI wrapper), risk_calc.c (TTC), fsm_controller.c (state machine), preprocess.c (normalization).

Benefit: Significantly improves code readability and long-term maintainability; grouped in Keil via *Add Group*.



| Memory Configuration (Stack & Heap)

Keeping main.c clean through modular design

Why it matters: ML activation buffers and temporary variables need generous RAM; functions like `snprintf` also require adequate stack space.

Stack Size: 0x2000 (8 KB)

Heap Size: 0x4000 (16 KB)

Result: Sized specifically to avoid stack/heap overflow errors during inference and UART logging.

Option	Value
Stack Configuration	
Stack Size (in Bytes)	0x0000 2000
Heap Configuration	
Heap Size (in Bytes)	0x0000 4000

| Peripheral Interfaces: SPI Display & UART Debug

Getting visual and textual feedback out of the MCU

LCD (ILI9341) over SPI1: 8-bit, MSB-first, Prescaler 2, CPOL Low / CPHA 1st Edge. PA5→SCK, PA7→MOSI, PA4→CS, PA6→DC, PA8→RST.

UART Debug (USART2): 9600 baud, 8 data bits, no parity, 1 stop bit. PA2→TX, PA3→RX, connected to Proteus's Virtual Terminal.

Interrupt-driven RX: USART2 interrupt registered in NVIC so incoming data doesn't block the main loop.

| AI Inference Engine & Preprocessing Pipeline

From raw pixels to a probability score

Test Input Strategy: Since a direct camera-to-MCU link isn't feasible in Proteus, 5 sample 96×96 images are converted to hexadecimal C arrays (via a Python script) and stored in Flash as test vectors.

Normalization: Preprocess_NormalizeFloat converts raw pixel bytes to float values before inference.

Inference Call: infer_engine.c invokes the Cube.AI-generated ai_ob_model_run() function.

Output: A float between 0–1, multiplied by 100 to report an obstacle probability percentage.

 CreateIMG.py	۱۴۰۵/۰۳/۲۶ ظ.ب ۱۰:۲۲	Python Source File	1 KB
 image_data.txt	۱۴۰۵/۰۴/۱۳ ظ.ب ۱۰:۱۸	Text Document	56 KB
 obs_0.png	۱۴۰۵/۰۳/۱۶ ظ.ق ۰۱:۱۹	PNG File	6 KB

| Risk Assessment (TTC) & FSM Decision Logic

Turning a probability into a driving decision

TTC Formula: $TTC = (\text{Distance} \times 1000) / \text{Speed}$, computed in `risk_calc.c` from simulated distance/speed values.

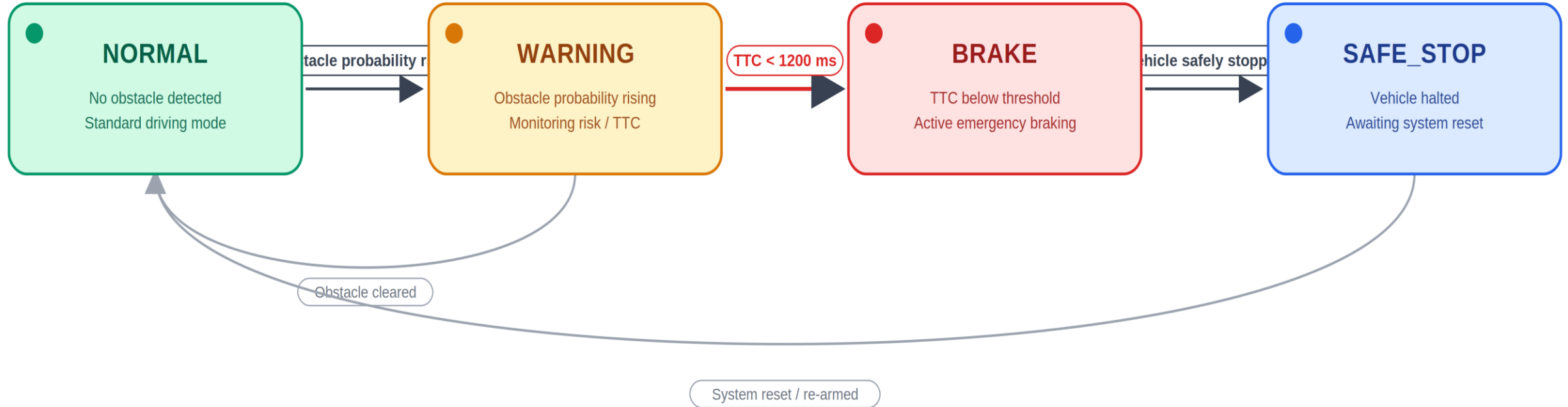
Four States: NORMAL → WARNING → BRAKE → SAFE_STOP, implemented in `fsm_controller.c`.

Trigger Condition: If the system is in WARNING and TTC drops below 1200 ms, it immediately transitions to BRAKE.

Display Sync: The LCD updates its status text/color in `main.c` in lockstep with the current FSM state.

FSM Decision Logic — Node 5 (TinyOD-Brake)

State transitions driven by obstacle probability and Time-to-Collision (TTC)



* Only WARNING → BRAKE (TTC < 1200 ms) is explicitly defined in the project report. Other transitions reflect typical FSM logic — verify against fsm_controller.c.

Integration, Emergency Scenarios & Proteus Simulation

PHASE 3

In this phase, all modules developed in previous phases (artificial intelligence, control logic and display drivers) were integrated and tested on the virtual hardware of the VE401F32STM microcontroller in the Proteus environment. The aim of this phase is to evaluate the correctness of the performance of the entire system in a simulated environment before implementing it on a real board.

| Schematic Design & Hardware Wiring

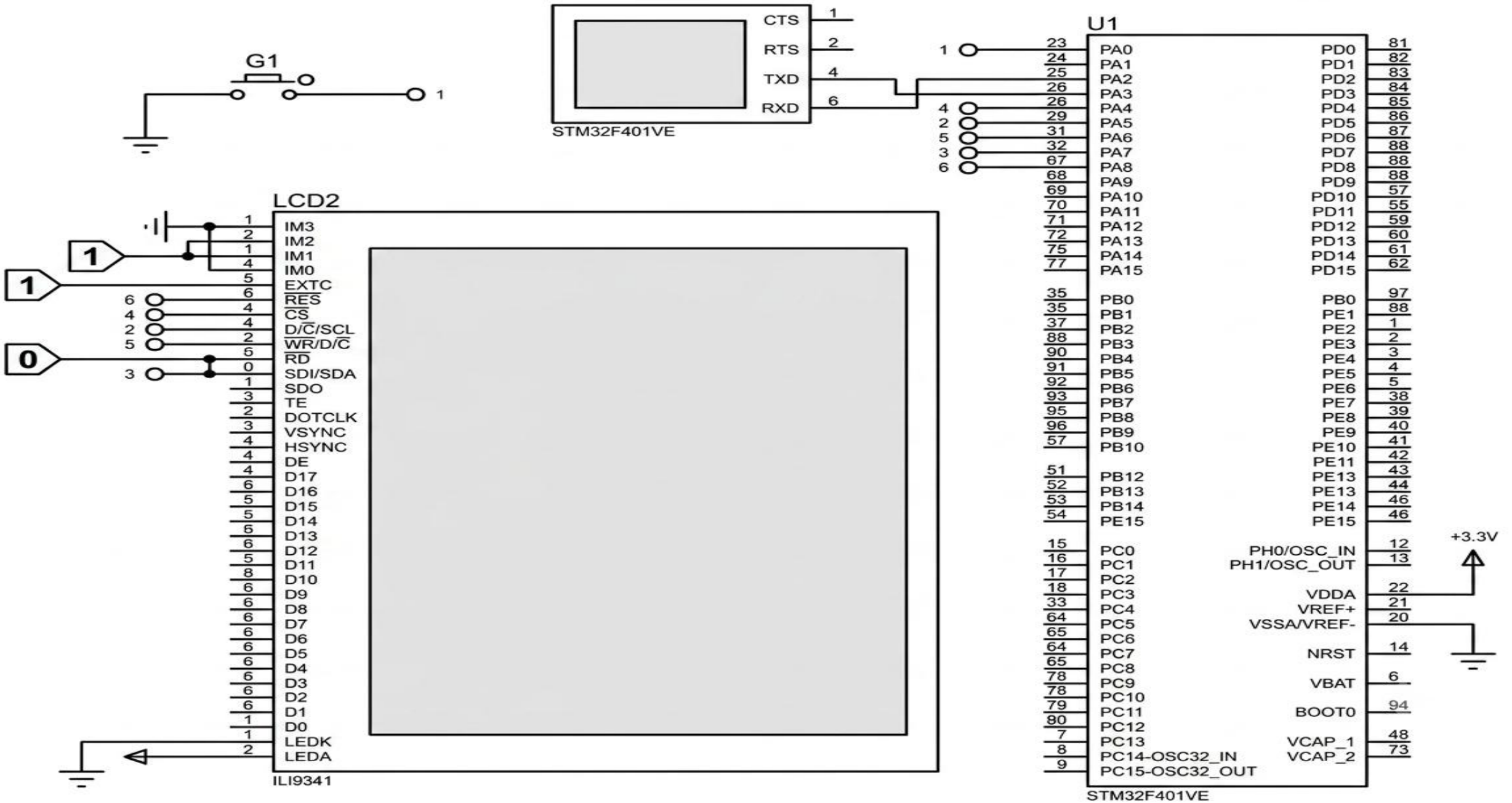
Assembling the virtual prototype in Proteus

MCU: STM32F401VE placed as the central processing unit.

Debug Link: UART TX/RX (PA2/PA3) wired to Proteus's Virtual Terminal for real-time monitoring.

Display: ILI9341 color LCD wired via SPI, with mode-select pins (IM0–IM3) and control lines (CS, RES) tied to MCU GPIOs.

Input: A push-button used to step through the 5 test scenarios.



| System Initialization & Boot Sequence

What happens the moment simulation starts

Model Load: Activation memory (Act Size: 41,188 bytes) and weight memory (Weights Size: 3,012 bytes) are allocated at boot.

Confirmation: The terminal prints AI Engine Initialized Successfully!, confirming the model loaded correctly into MCU memory.

Ready State: The system then enters its main loop, continuously running inference on the current simulated frame.

Virtual Terminal

```
Act Size: 41188, Weights Size: 3012
AI Engine Initialized Successfully!
AI Engine Run | In Size: 9216 | Prob: 8% | Inference OK | Time: 116 ms
State: 0
AI Engine Run | In Size: 9216 | Prob: 8% | Inference OK | Time: 115 ms
State: 0
AI Engine Run | In Size: 9216 | Prob: 8% | Inference OK | Time: 120 ms
State: 0
AI Engine Run | In Size: 9216 | Prob: 44% | Inference OK | Time: 117 ms
State: 0
AI Engine Run | In Size: 9216 | Prob: 67% | Inference OK | Time: 113 ms
State: 1
AI Engine Run | In Size: 9216 | Prob: 67% | Inference OK | Time: 119 ms
State: 1
AI Engine Run | In Size: 9216 | Prob: 79% | Inference OK | Time: 118 ms
State: 2
AI Engine Run | In Size: 9216 | Prob: 79% | Inference OK | Time: 121 ms
State: 3
AI Engine Run | In Size: 9216 | Prob: 95% | Inference OK | Time: 122 ms
State: 3
AI Engine Run | In Size: 9216 | Prob: 95% | Inference OK | Time: 118 ms
State: 3
```

| Scenario Test: Normal / Safe Condition

Baseline behavior with no obstacle present

Input: Simulated frame with background noise only, no obstacle.

AI Output: 8% obstacle probability.

System Response: FSM remains in NORMAL;
LCD displays a green **"Safe"** status.



| Scenario Test: Emergency Braking

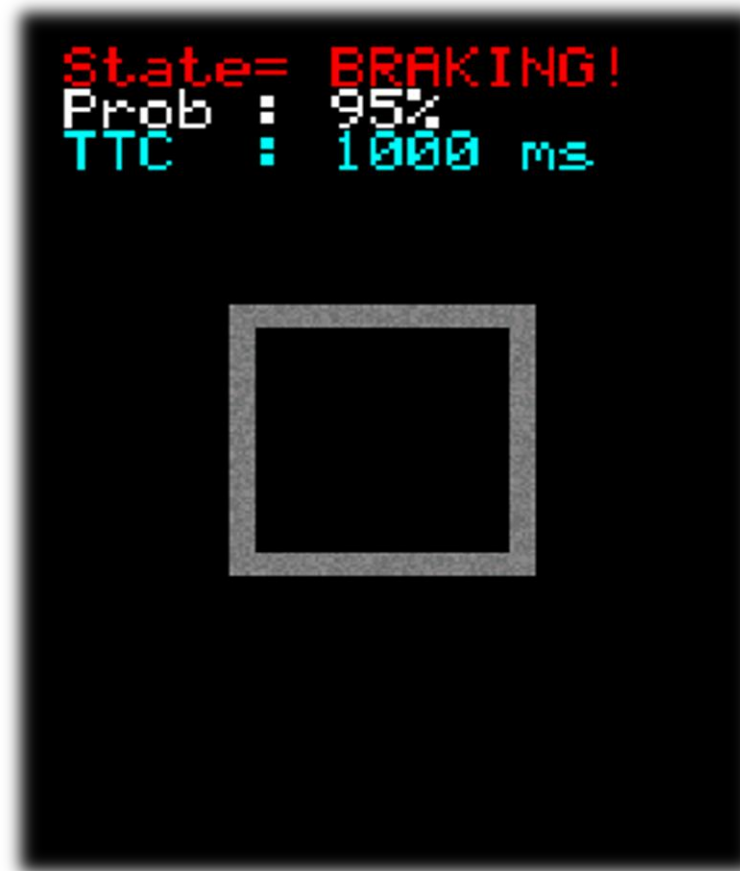
Reaction to a closing obstacle

Trigger: Pressing the input button simulates an approaching obstacle.

AI Output: Obstacle probability jumps to 95%.

Risk Calculation: TTC computed at 1000 ms — below the 1200 ms braking threshold.

System Response: FSM immediately transitions to BRAKING!; LCD switches to a red warning display.



| Performance Evaluation

Measured, not estimated, timing

Measured Inference Time: 115–122 ms per cycle, read directly from on-chip timing logged over UART.

Effective Frame Rate: Roughly 8–9 FPS at this clock configuration.

Assessment: This confirms the pipeline is functionally correct end-to-end, but the current frame rate is on the low side for a safety-critical automotive use case and would benefit from further optimization (e.g., a higher system clock or a lighter model).

| Conclusion & Toolchain Summary

From idea to a fully simulated edge-AI pipeline

Outcome: A complete obstacle-detection and emergency-braking pipeline was designed, trained, deployed, and validated entirely through simulation — no physical hardware required.

Toolchain: Python/PyTorch (model design & training) → STM32Cube.AI & STM32CubeMX (model-to-C conversion, hardware setup) → Keil MDK (firmware) → Proteus (full-system simulation).

Build Health: Final firmware compiled with 0 errors, 0 warnings.

```
Rebuild started: Project: TinyML_OD_F4
*** Using Compiler 'V6.24', folder: 'D:\Keil\ARM\ARMCLANG\Bin'
Rebuild target 'TinyML_OD_F4'
assembling startup_stm32f401xe.s...
compiling crc.c...
compiling stm32f4xx_hal_rcc_ex.c...
compiling ili9341.c...
compiling stm32f4xx_hal_flash_ex.c...
compiling stm32f4xx_it.c...
compiling stm32f4xx_hal_msp.c...
compiling stm32f4xx_hal_crc.c...
compiling spi.c...
compiling uart.c...
compiling gpio.c...
compiling stm32f4xx_hal_flash.c...
compiling main.c...
compiling stm32f4xx_hal_rcc.c...
compiling stm32f4xx_hal_flash_ramfunc.c...
compiling stm32f4xx_hal_gpio.c...
compiling stm32f4xx_hal_pwr.c...
compiling stm32f4xx_hal_pwr_ex.c...
compiling system_stm32f4xx.c...
compiling stm32f4xx_hal_dma_ex.c...
compiling stm32f4xx_hal_exti.c...
compiling stm32f4xx_hal_cortex.c...
compiling stm32f4xx_hal.c...
compiling stm32f4xx_hal_dma.c...
compiling sensor_sim.c...
compiling risk_calc.c...
compiling infer_engine.c...
compiling fsm_controller.c...
compiling preprocess.c...
compiling ob_model_data_params.c...
compiling ob_model_data.c...
compiling stm32f4xx_hal_spi.c...
compiling stm32f4xx_hal_uart.c...
compiling ob_model.c...
linking...
Program Size: Code=19608 RO-data=4296 RW-data=3628 ZI-data=68764
FromELF: creating hex file...
"TinyML_OD_F4\TinyML_OD_F4.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:01
```

Ai node: ready for integration with the master.

END OF NODE 5

We sincerely thank you for honoring this presentation with your presence and attention.

With special thanks to our esteemed professor, Dr. Zargari,
for his invaluable guidance and unwavering support.

Presenting Group:

Parinaz Kari | Hossein Ranjbari | Sobhan Nasiri

Salar Ghasemi | Mahdi Tahouni